BPMS - WfMS

Prof. Dr. Andreas Hense
andreas.hense@h-brs.de

Wirtschaftsinformatik

Hochschule
Bonn-Rhein-Sieg
University of Applied Sciences

Agenda I

1. Workflow Management Systeme
   - Definition
   - Referenzmodell
2. Workflow Patterns
   - Einführung
   - Kontrollfluss Patterns
   - Weitere Patterns
   - Problem XOR-Join
3. Yet Another Workflow Language
   - Einführung
   - Anwendungsbeispiel
   - Kontrollfluss Perspektive
   - XPATH
   - XQUERY
   - Daten Perspektive
   - Ressourcen Perspektive

Agenda II

- Resource Service
- Methode zum Erstellen eines Workflows

4. Literatur

Workflow Management Systeme

Definition

Definition

Ein Workflow Management System (WfMS) ist ein generisches Software Werkzeug, mit welchem man Workflows definieren, ausführen, registrieren und kontrollieren kann. [Law97]

Definition

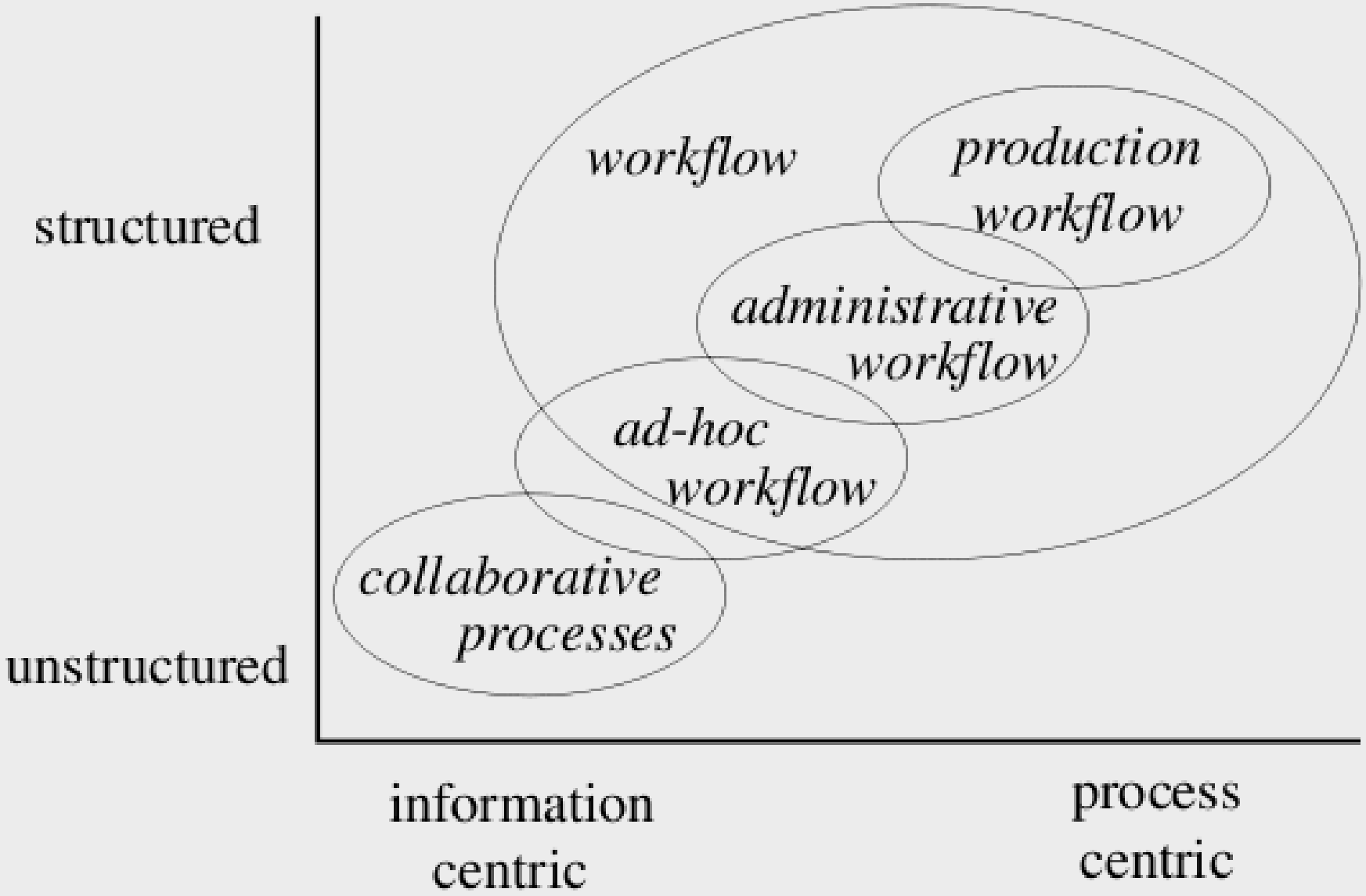
Workflow Management System: A system that defines, creates and manages the execution of workflows through the use of software, running on one or more workflow engines, which is able to interpret the process definition, interact with workflow participants and, where required, invoke the use of IT tools and applications. [WFM99]

- Seit 2004 wird verstärkt der Begriff Business Process Management System (BPMS) benutzt.
- Geschäftsprozess und Workflow sind zwei unterschiedliche Begriffe.
- Allerdings ist der Unterschied zwischen BPMS und WfMS in der Praxis gering – daher hier synonym.

Workflow Management Systeme

Referenzmodell

Workflow und Kollaboration



[Aal98]

Werkzeuge zur Kollaboration

Live Feedback

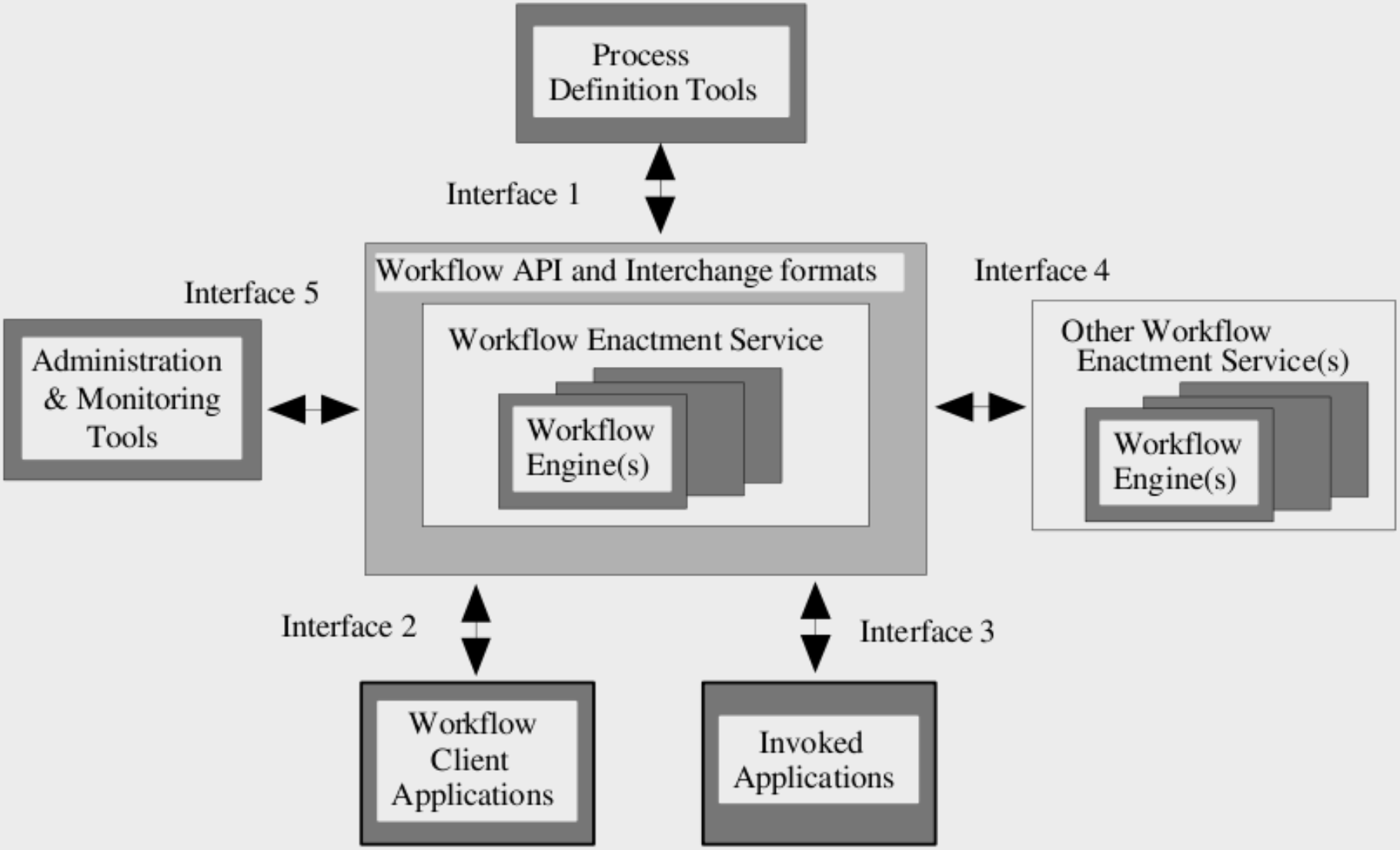
- Welche Werkzeuge, die die Zusammenarbeit von Teams unterstützen, kennen Sie?

Abgrenzung Groupware I

- Collaborative Processes werden von Groupware oder auch Collaborative Software unterstützt.
- Funktionalitäten umfassen Projektmanagement, Kalender, eMail, Chat, Wiki, gemeinsame Verzeichnisse, etc.
- Kleine – nicht repräsentative – Auswahl mit unterschiedlichen Funktionalitäten:

  Slack Kommerziell.
  Microsoft Sharepoint Kommerziell.
  HCL Connections Kommerziell.
  Liferay Open-source Portal.
  Atlassian Confluence Kommerziell.
  Google Workspace Kommerziell.

Workflow Reference Model



[WFM99]

Vorteile Prozessautomatisierung mit WfMS [AAHR09]

Konformität Prozesse laufen konform zum Prozessmodell.

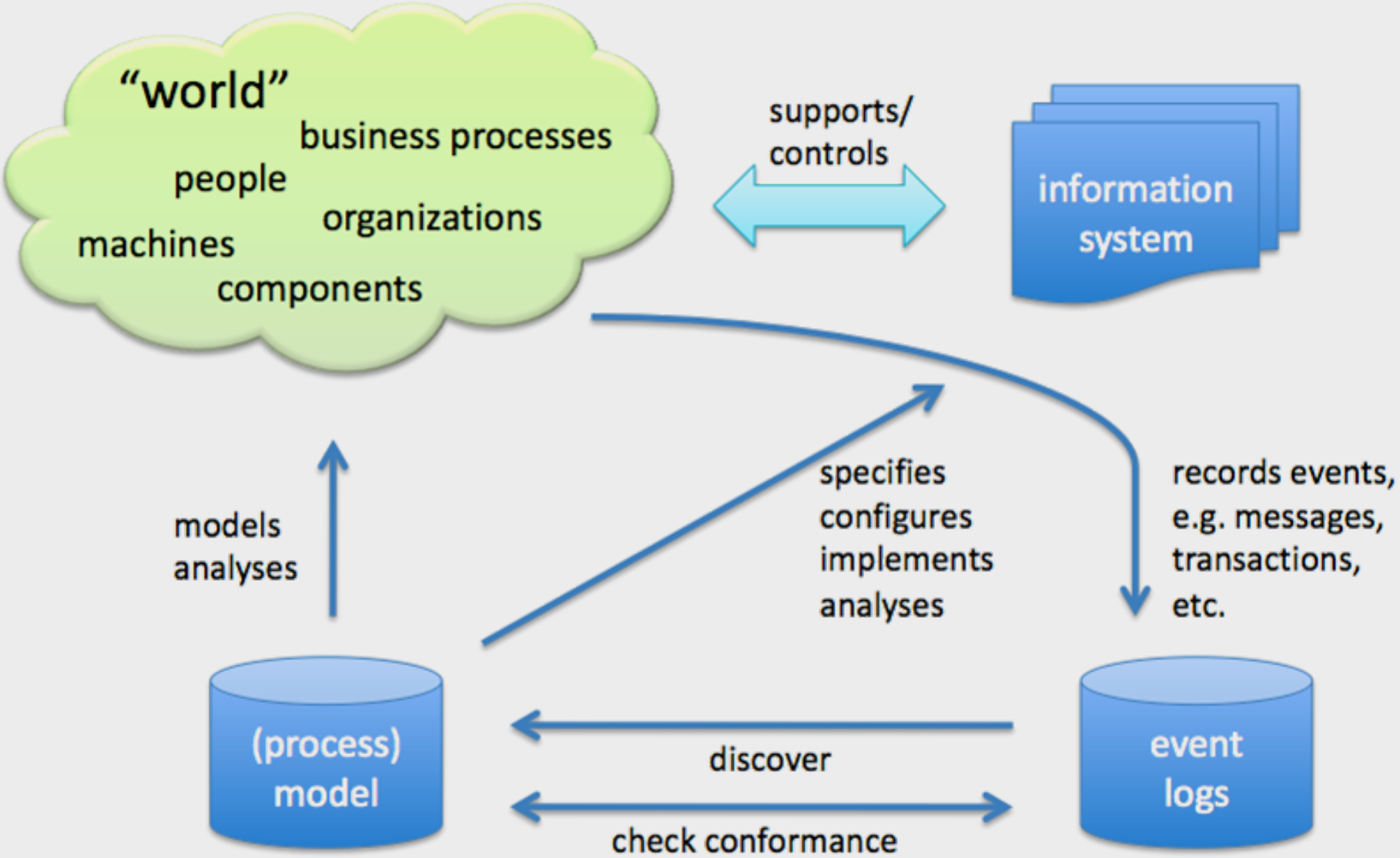
Explizite Repräsentation des Kontrollflusses Änderungen im Prozessmodell ziehen keine Implementierungsaufwand auf niedriger technischer Ebene nach sich.

Explizite Repräsentation der Ressourcen Die Arbeit kann den richtigen Ressourcen zugewiesen werden und die Arbeitsbelastung oder die Historie kann bei der Zuweisung berücksichtigt werden.

Kopplung von Prozessen mit Daten unterstützt die Korrektheit der Daten.

Unterstützung beim Monitoring Identifikation und Lösung von Engpässen.

Process Mining Analyse nach der Ausführung von Workflows zur Prozessverbesserung.


"world"
business processes
people
organizations
machines
components
supports/
controls
information
system
models
analyses
specifies
configures
implements
analyses
records events,
e.g. messages,
transactions,
etc.
(process)
model
discover
check conformance
event
logs

[AAHR09]

Workflow Patterns Initiative

- Vor dem Jahr 2000: Vielzahl von WfMS.
  - nicht standardisiert.
  - keine formale Semantik.
- Daher Gründung der Workflow Patterns Initiative. [AH10]
  - Unparteiischer Vergleich von WfMS und Basis für die Verbesserung der Ansätze.
  - Geeignetheit sollte Vorrang vor reiner Ausdrucksstärke haben.
  - Zwischen 1999 und 2006 Vergleich bestehender WfMS und Prozessmodellierungssprachen.
- Ein Ergebnis dieser Arbeit war die auf Petrinetz-Semantik basierende Sprache YAWL.

Workflow Patterns

Einführung

Drei Perspektiven

Die Workflow Patterns sind nach den folgenden drei Perspektiven aufgebaut [RAH16]:

Kontrollfluss beschreibt die Tasks eines Geschäftsprozesses, in welcher Reihenfolge diese ausgeführt werden können und wie die einzelnen Aktivitäten ermöglicht werden.

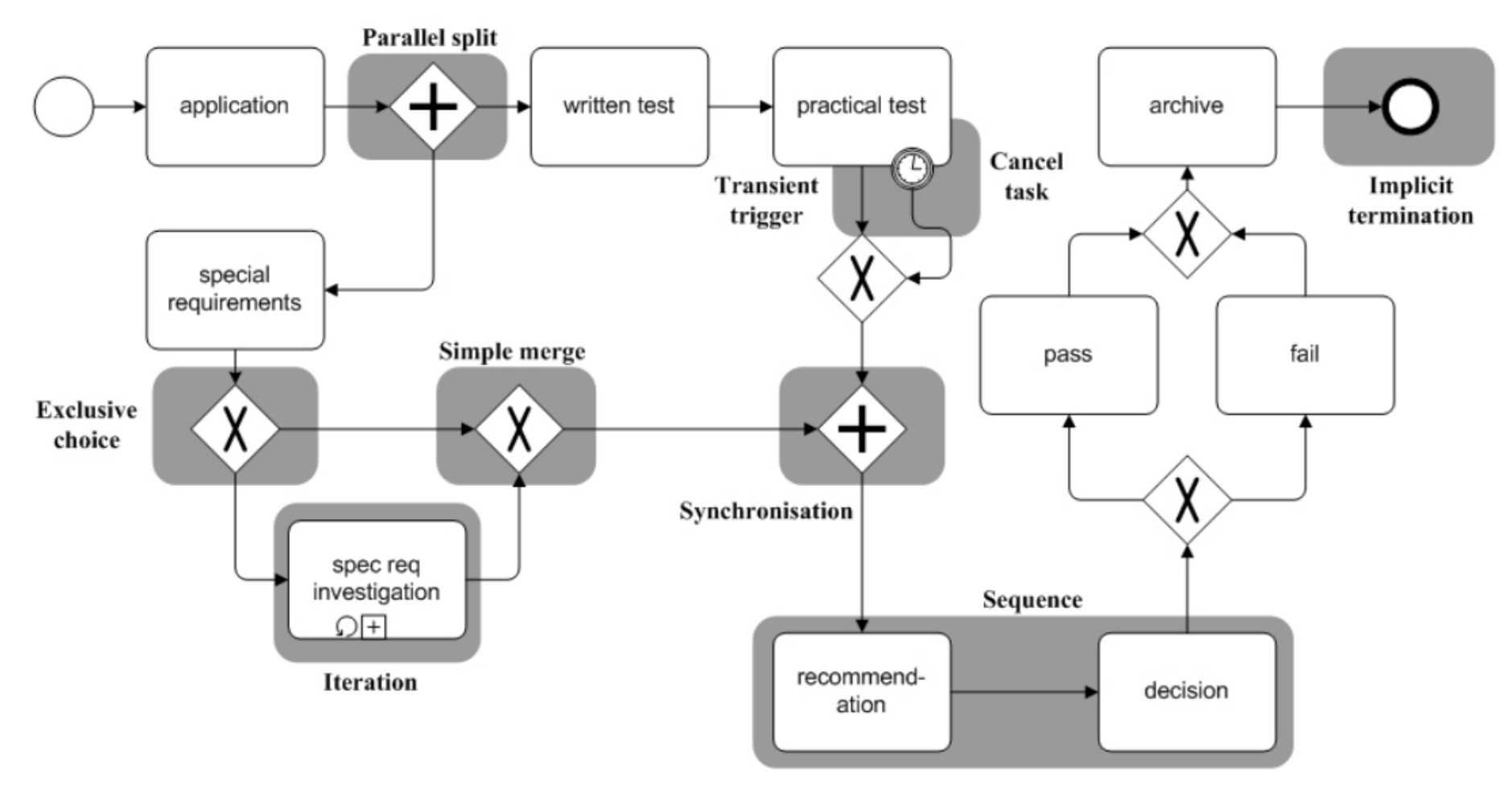
Daten beschreibt die Informationen, welche während der Ausführung eines Geschäftsprozesses verarbeitet werden.

Ressourcen beschreibt die Personen und technischen Komponenten, die einen Geschäftsprozess ausführen und die Beziehungen zwischen diesen.

Workflow Patterns

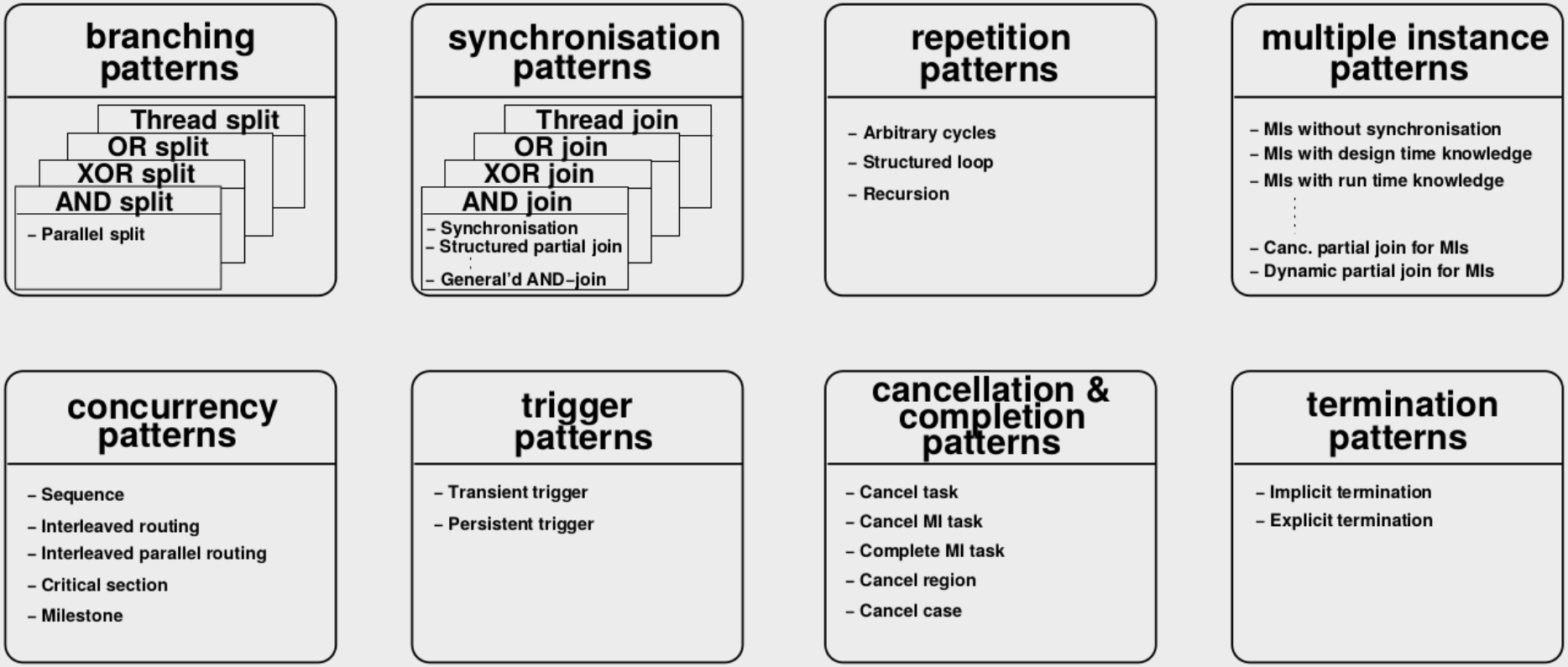
Kontrollfluss Patterns

Beispiel für Patterns in BPMN



[RAH16]

Übersicht Kontrollfluss Patterns



[RAH16]

AND-Split

Beschreibung Verzweigung auf zwei oder mehr parallele Stränge, die jeweils nebenläufig ausgeführt werden.

Beispiel Nachdem der Kunde für die Ware bezahlt hat, wird die Ware verpackt und eine Quittung ausgestellt. Die letzten beiden Aktivitäten können simultan oder in jeder beliebigen Reihenfolge durchgeführt werden.

Motivation Die parallele Durchführung von Vorgängen ermöglicht Flexibilität und ein zügigeres Beenden des Prozesses. Die parallelen Stränge können – müssen aber nicht – später wieder synchronisiert werden.

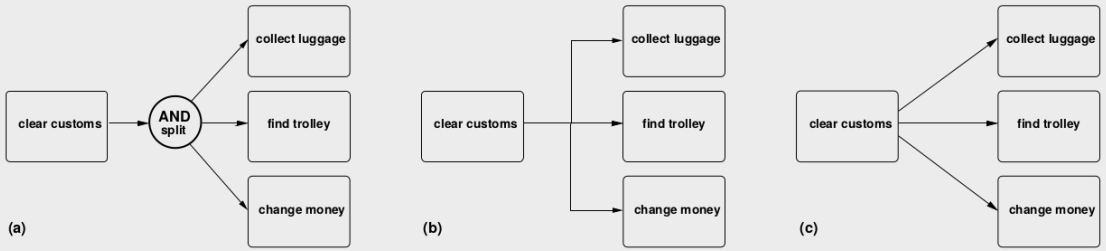
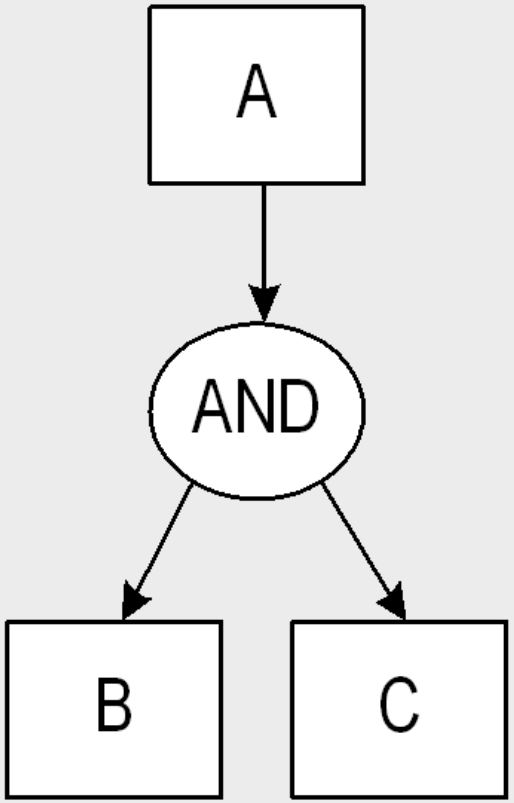


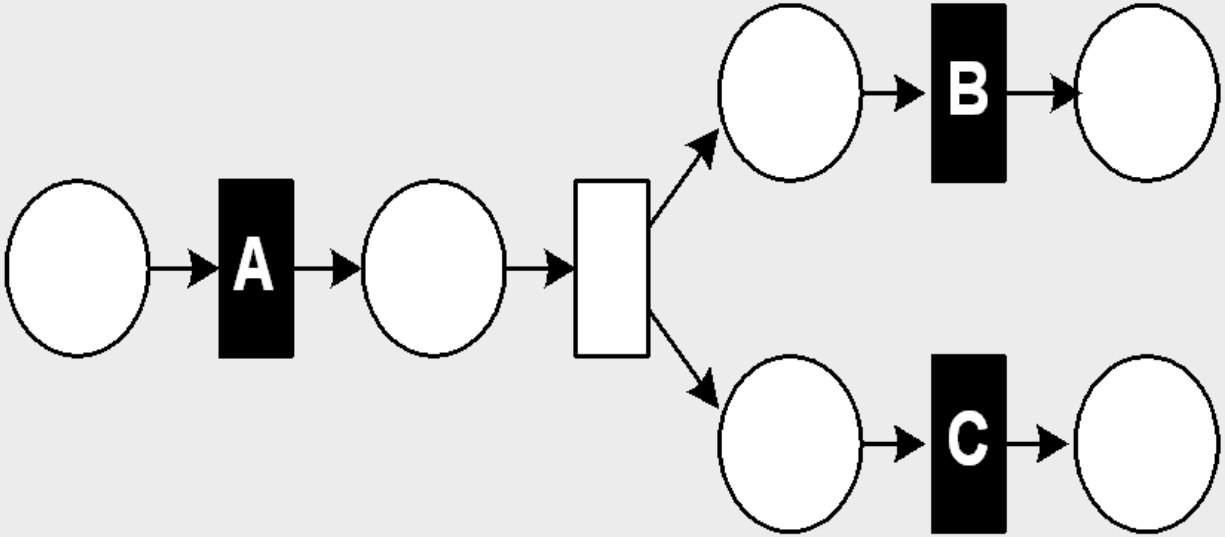
Figure 4.4: Parallel split operation – (a) explicit depiction and (b), (c) implicit depiction

[RAH16]

AND-Split mit Petrinetz



[AAHR09]



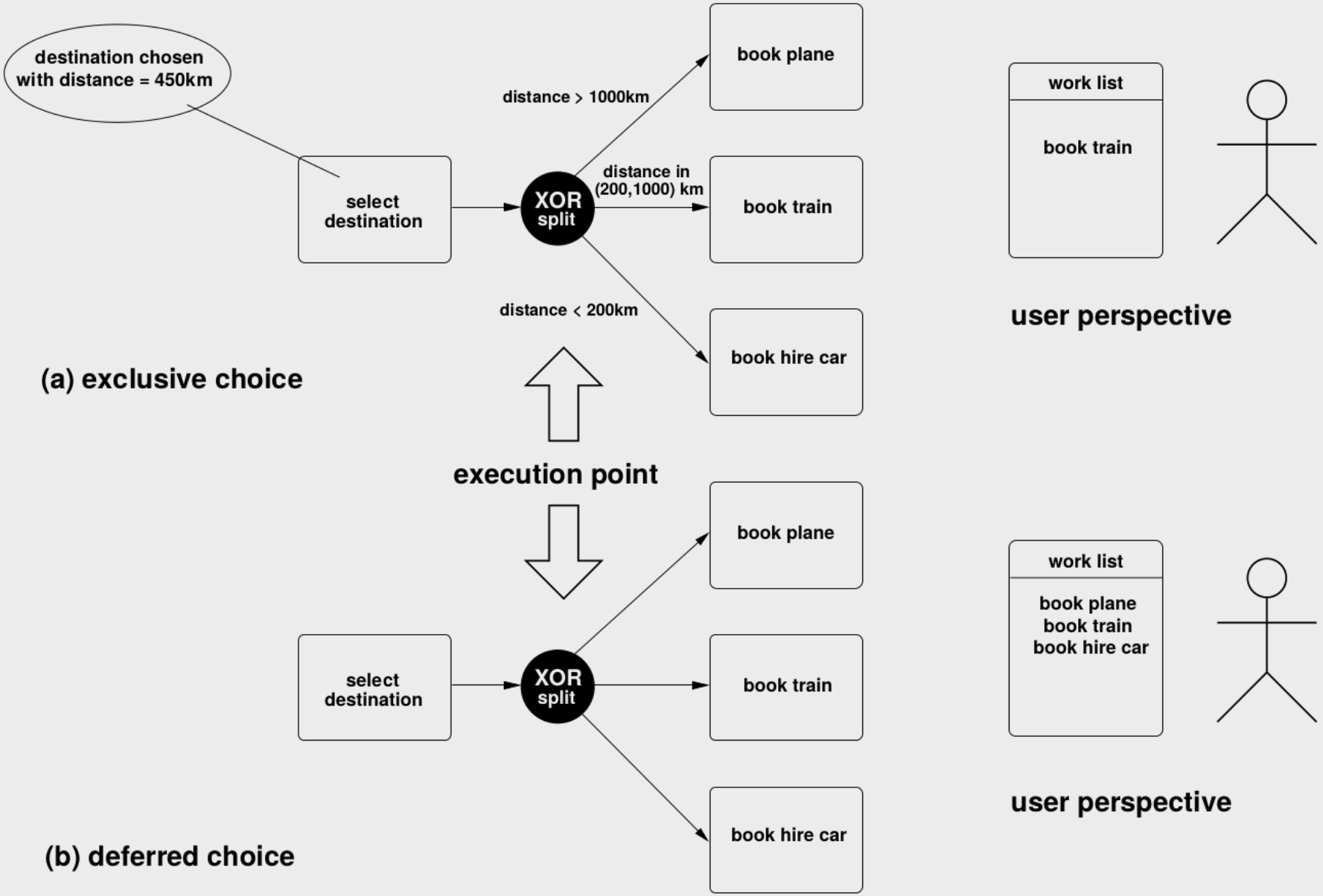
XOR-Split – Exclusive Choice

Beschreibung Verzweigung in zwei oder mehr Stränge, von denen genau eine sofort ausgewählt wird.

Beispiel Nachdem die Ausschreibungssumme festgelegt wurde, entscheidet das System automatisch anhand der Höhe der Summe, ob eine öffentliche (größer gleich 200.000) oder eine begrenzte (kleiner 200.000) Ausschreibung durchgeführt wird.

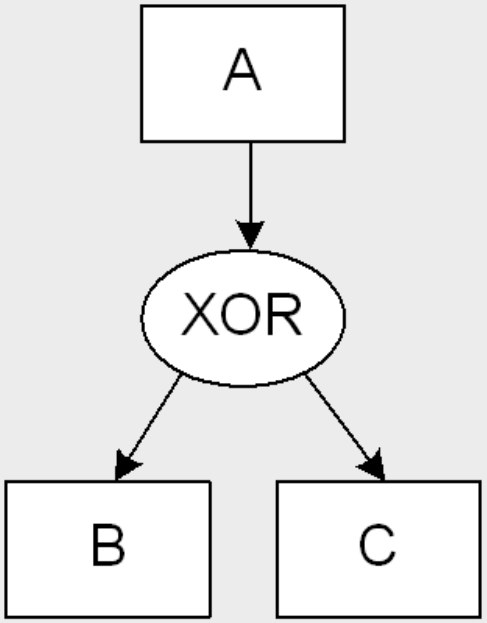
Motivation Dieses Muster erlaubt zu spezifischen Tasks zu verzweigen in Abhängigkeit von Datenelementen im Prozess. Diese werden in Ausdrücken ausgewertet. Die Entscheidung über den weiteren Fortgang wird dynamisch getroffen.

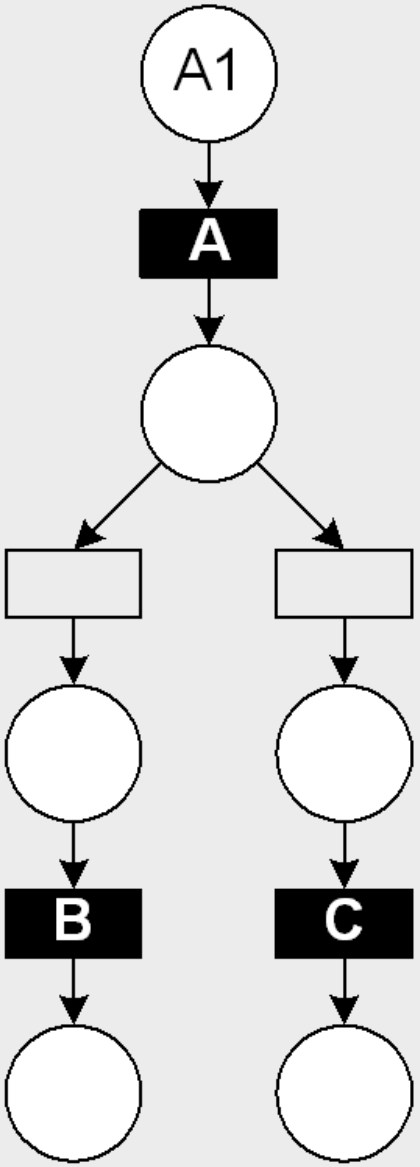
Die beiden Möglichkeiten eines XOR-Splits

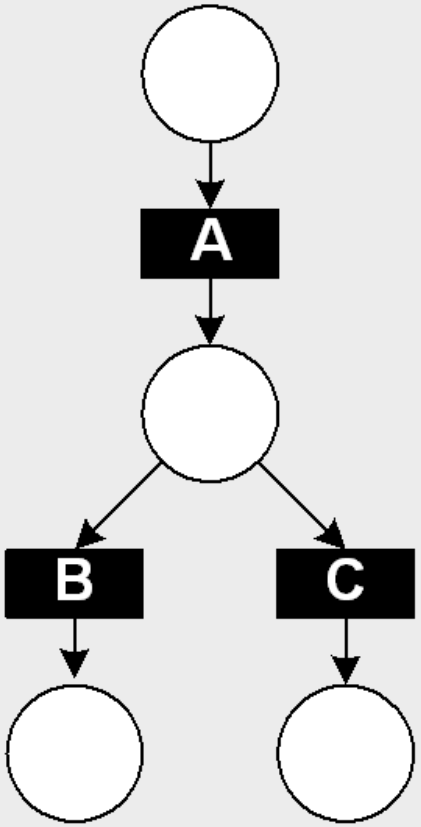


[RAH16]

Die beiden Möglichkeiten eines XOR-Splits als Petrinetz







Explicit choice and deferred choice!! [AAHR09]

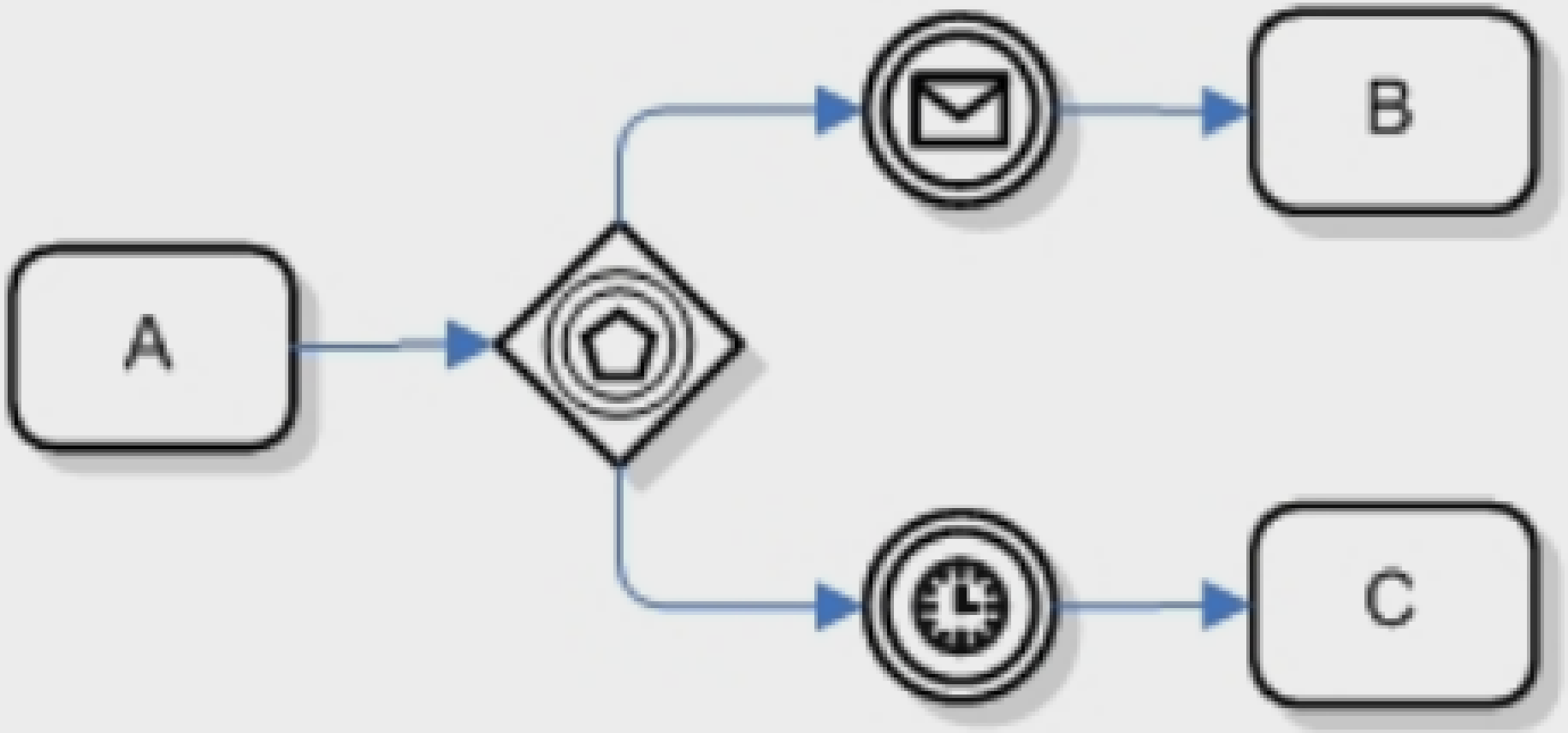
XOR-Split – Deferred Choice

Beschreibung Der Punkt in einem Prozess, wo eine von mehreren Möglichkeiten mittels einer Interaktion mit der Umgebung ausgewählt wird – meist eine Person, die eine Möglichkeit aussucht. Nachdem eine Möglichkeit ausgesucht wurde, werden die anderen zurückgezogen.

Beispiel Nachdem der Kunde im Onlineshop zur Kasse gegangen ist, kann er eine von zwei Tasks auswählen: mit Kreditkarte zahlen oder per Banküberweisung zahlen.

Motivation Dieses Muster erlaubt, die Entscheidung bis zu allerletzt zu verschieben und nicht durch das Workflow-System sondern die Umgebung treffen zu lassen.

Deferred Choice in BPMN mit ereignisbasiertem Gateway



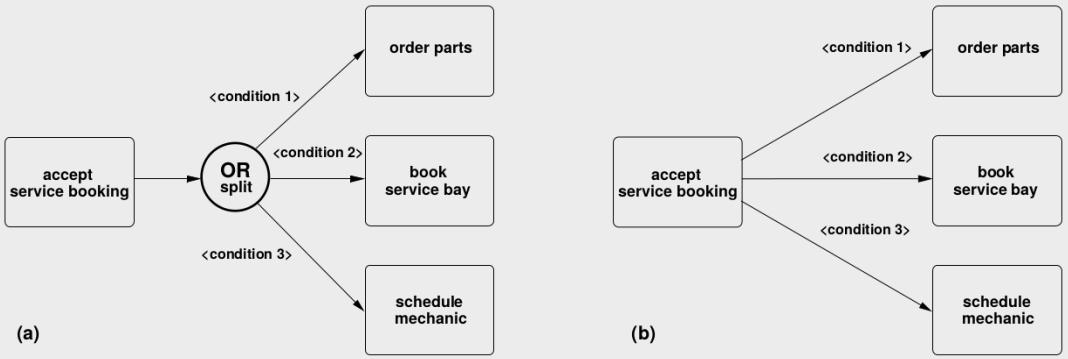
[RAH16]

OR-Split

Beschreibung Verzweigung in zwei oder mehr Stränge, von denen einer oder mehrere sofort ausgewählt wird.

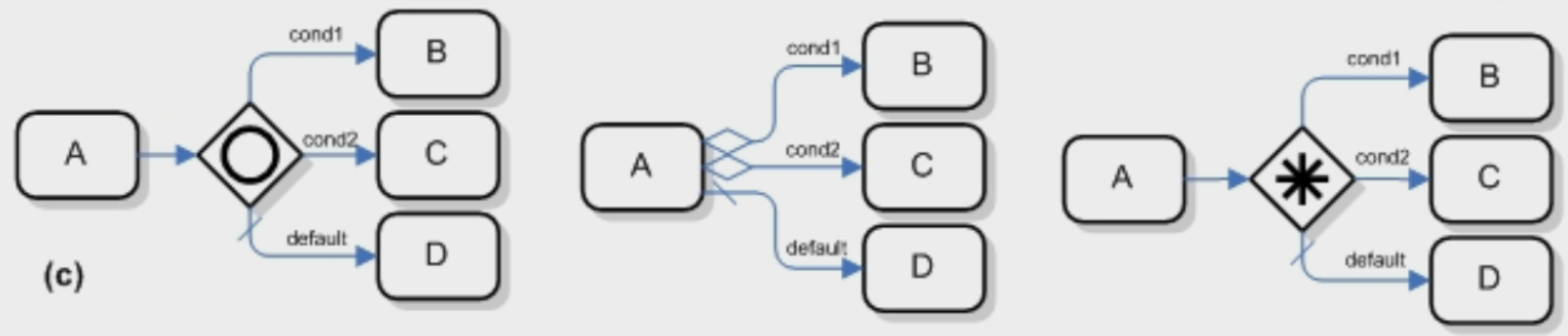
Beispiel Nachdem das Netzwerk ausgefallen ist, können die Tätigkeiten "Reboot Firewall", "Reset Router" und "Test Cable Connectivity" nach Bedarf ausgewählt werden.

Motivation Erlaubt in einem Workflow selektiv fortzufahren und die Entscheidung über die auszuführenden Wege dynamisch zu treffen.



[RAH16]

OR-Split bzw. Multi-Choice in BPMN



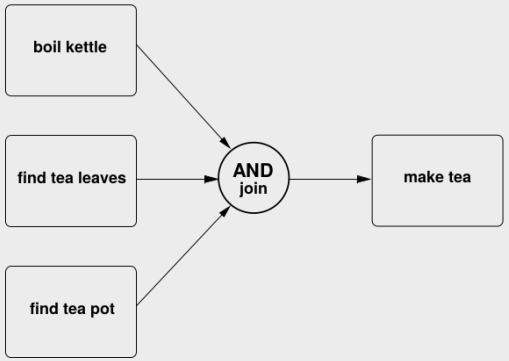
[RAH16]

AND-Join (Synchronisation)

Beschreibung Die Konvergenz zweier oder mehrerer Stränge in einen einzelnen Strang, so dass die Kontrolle weitergegeben wird, wenn alle Eingangsstränge aktiviert sind.

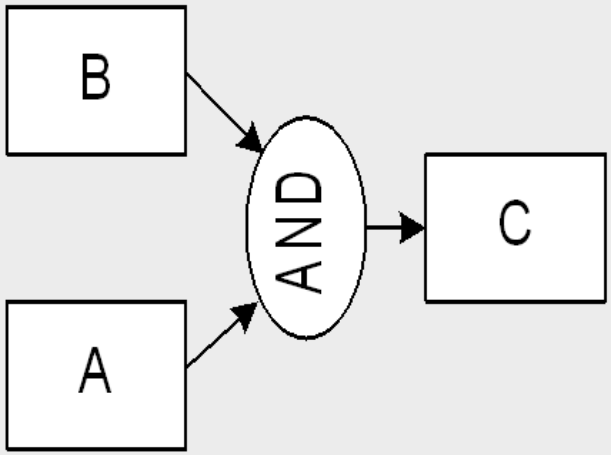
Beispiel Die Kasse kann erst dann abgeglichen werden, wenn der Laden geschlossen ist und die Kreditkartenübersicht gedruckt wurde.

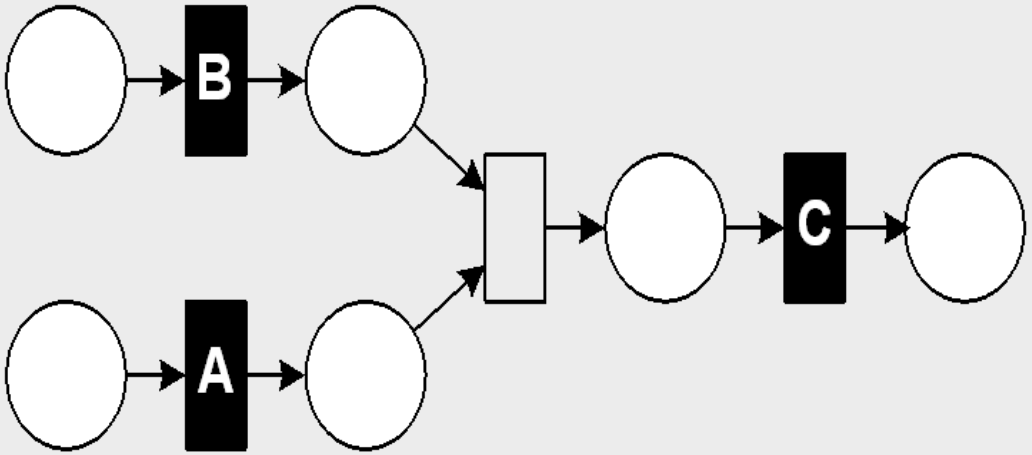
Motivation Erlaubt es, parallele Stränge wieder zusammenzuführen.



[RAH16]

AND-Join als Petrinetz





[AAHR09]

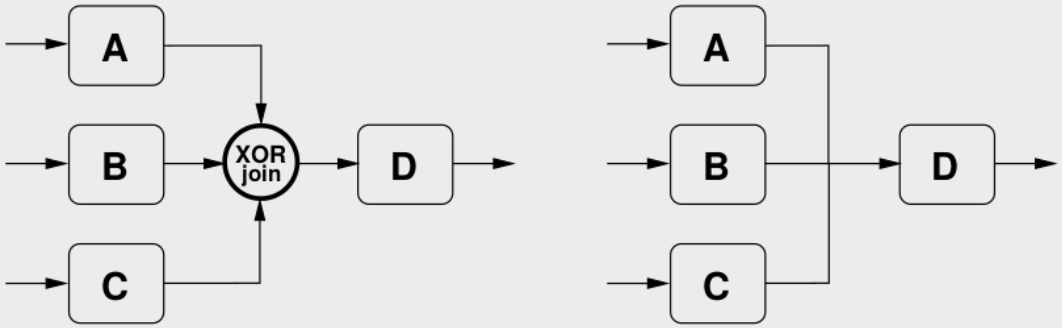
Synchronisation ist nur eine von mehreren Varianten des AND-Joins [RAH16].

XOR-Join (Simple Merge)

Beschreibung Die Konvergenz zweier oder mehrerer Stränge in einen einzelnen Strang, so dass die Kontrolle weitergegeben wird, jedesmal wenn einer der Eingangsstränge aktiviert ist.

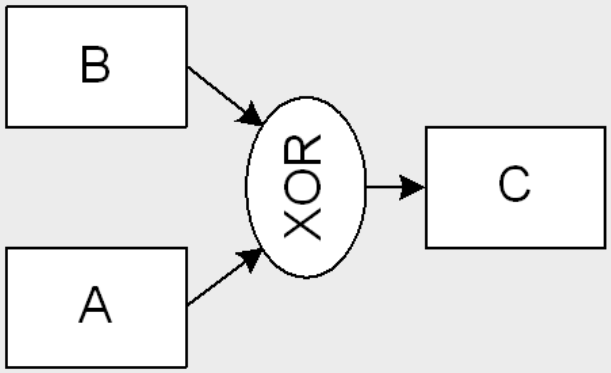
Beispiel Nach Erhalt der Zahlung in bar oder über Kreditkarte wird die Quittung gedruckt.

Motivation Erlaubt es, parallele Stränge ohne Synchronisierung zusammenzuführen. So kann die gemeinsame Weiterführung unterschiedlicher Stränge in einem Strang beschrieben werden.

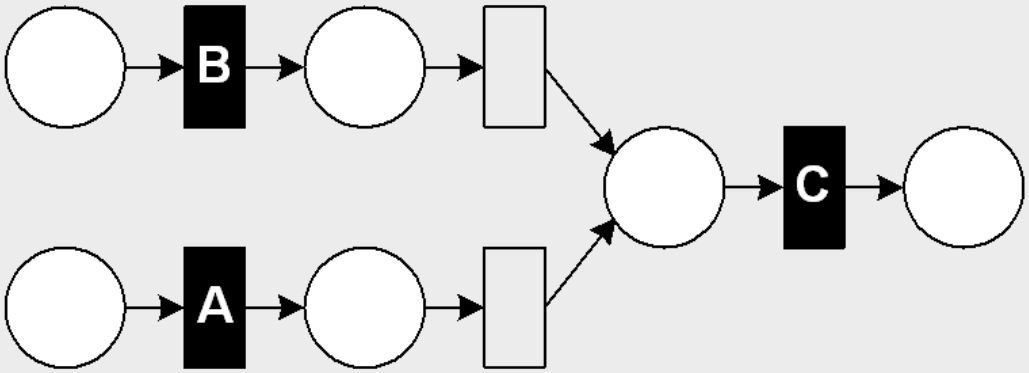


[RAH16]

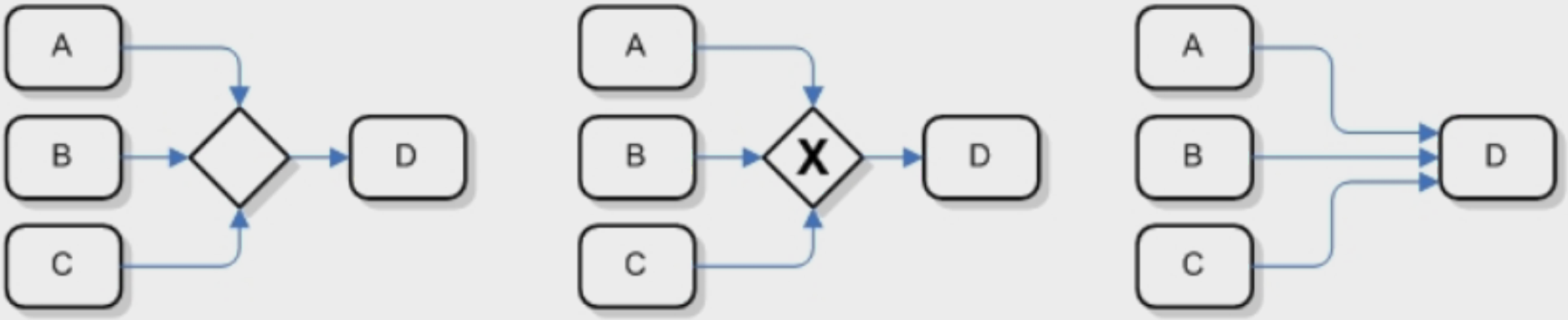
XOR-Join als Petrinetz



[AAHR09]



Simple Merge in BPMN



[RAH16]

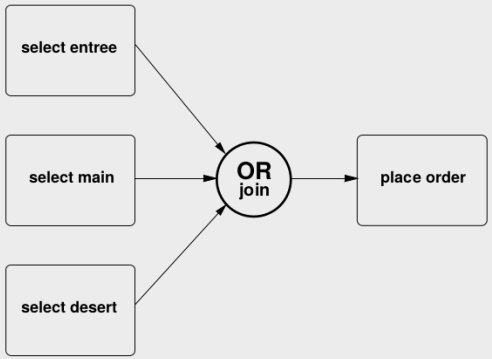
OR-Join (General synchronizing merge) I

Beschreibung Die Konvergenz zweier oder mehrerer Stränge in einen einzelnen Strang, so dass die Kontrolle weitergegeben wird, wenn mindestens einer der Eingangsstränge aktiviert ist und keiner von denen, die bisher nicht aktiviert sind, später aktiviert werden kann.

Beispiel Das Beispiel auf der nächsten Seite zeigt, dass im OR-join entschieden werden muss, ob die Schleife über "D" beendet wird. Dies kann nicht lokal entschieden werden.

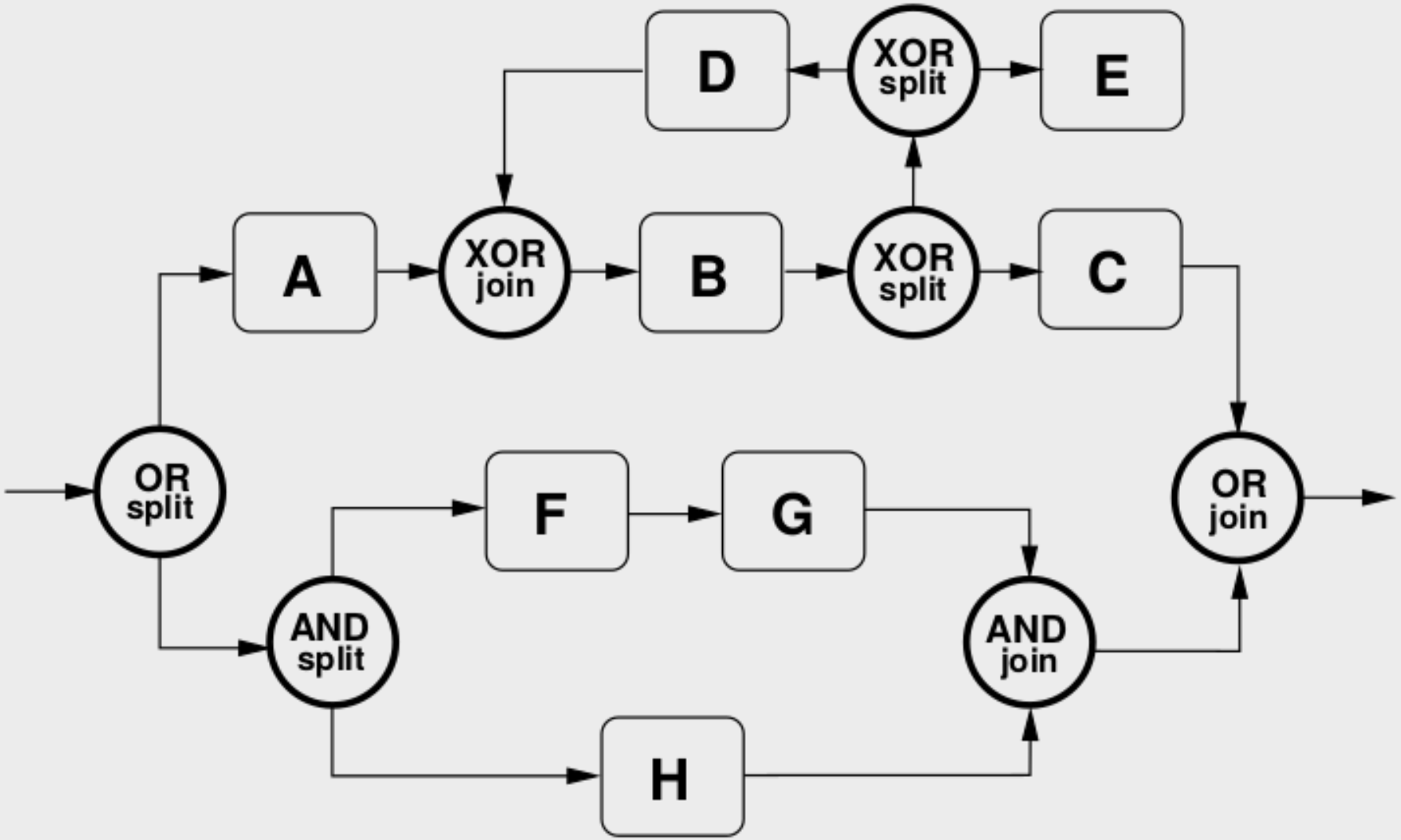
Motivation Erlaubt es, parallele Stränge ohne Synchronisierung zusammenzuführen. So kann die gemeinsame Weiterführung unterschiedlicher Stränge in einem Strang beschrieben werden.

OR-Join (General synchronizing merge) II



[RAH16]

Beispiel für General synchronizing merge

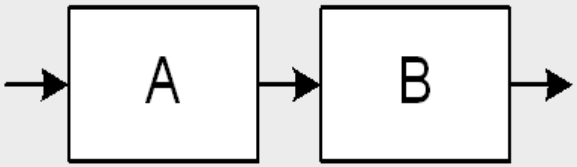


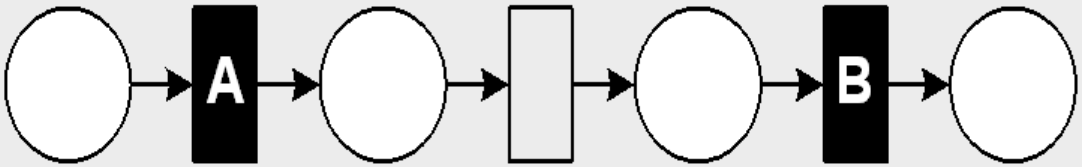
[RAH16]

Workflow Patterns

Weitere Patterns

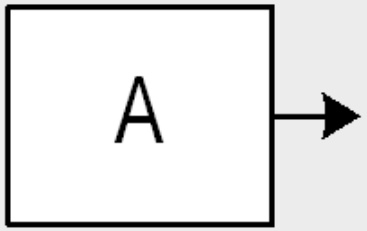
Sequence

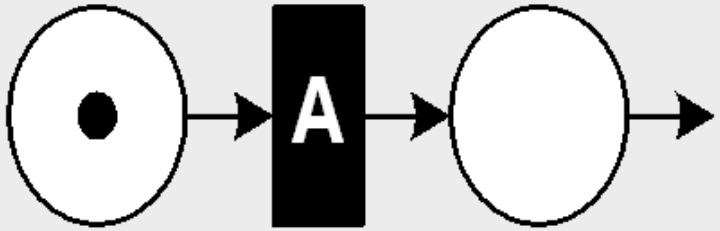


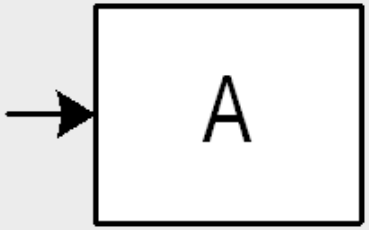


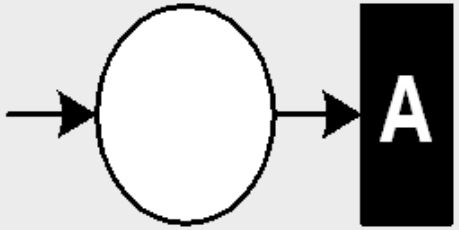
[AAHR09]

Initial/Final



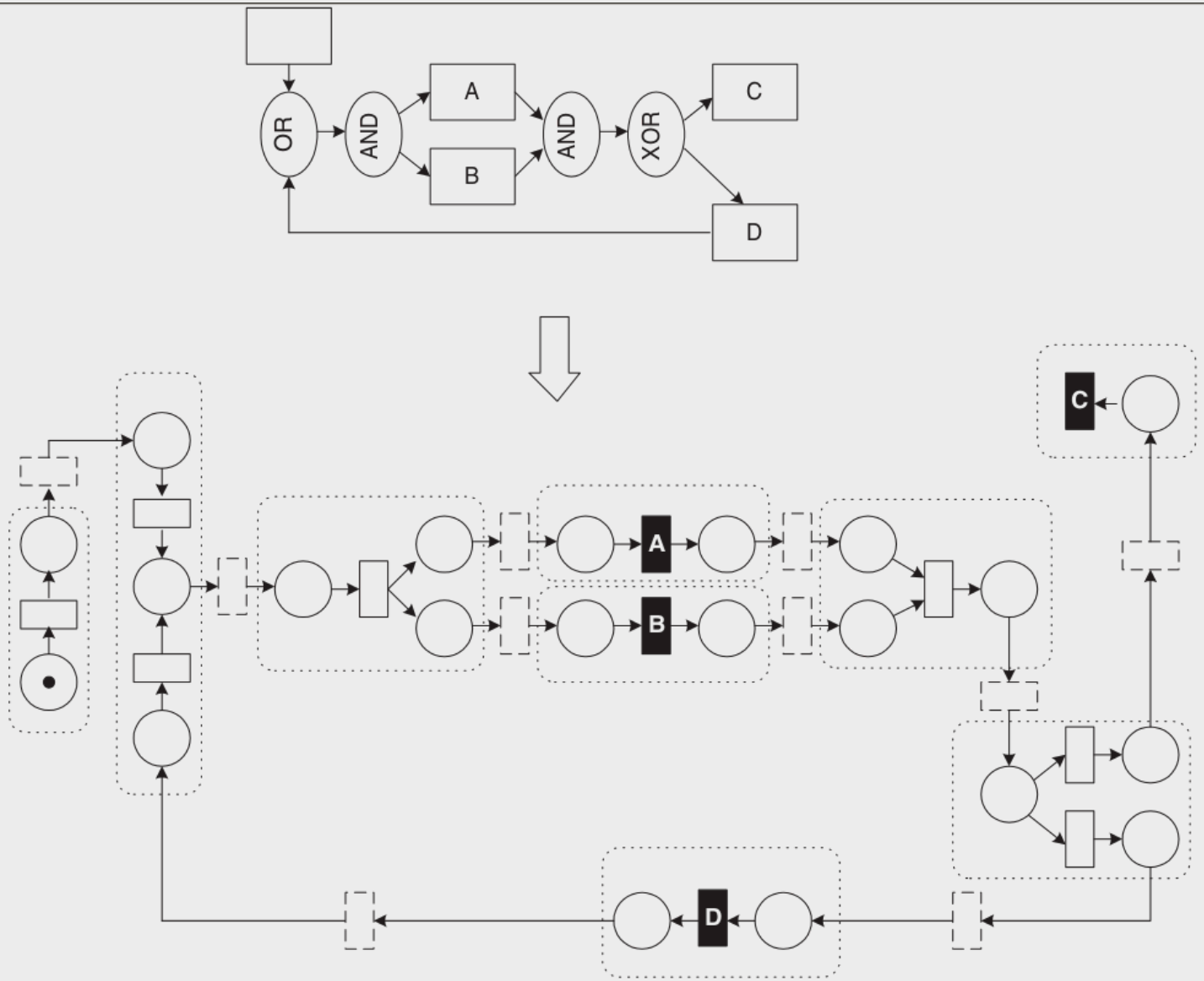






[AAHR09]

Translation of Sample Standard Workflow Model



[KHA02]

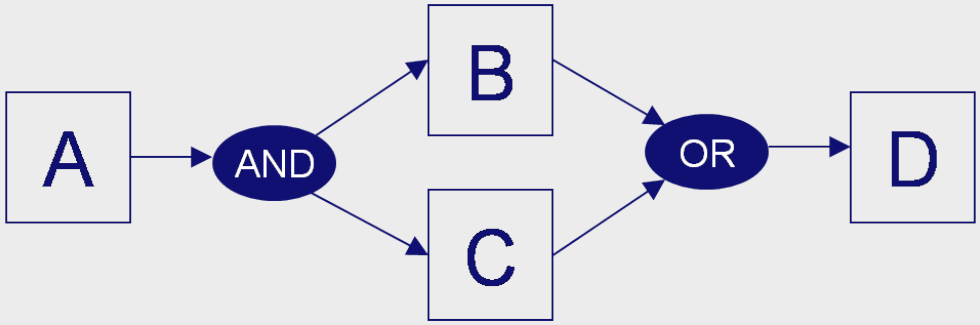
Komplexere Pattern I

- Weitere Workflow Pattern und deren Semantik unter [AH10].
- Dort befinden sich auch Flash-Animationen der Patterns.

Workflow Patterns

Problem XOR-Join

(X)OR-Join I



[AAHR09]

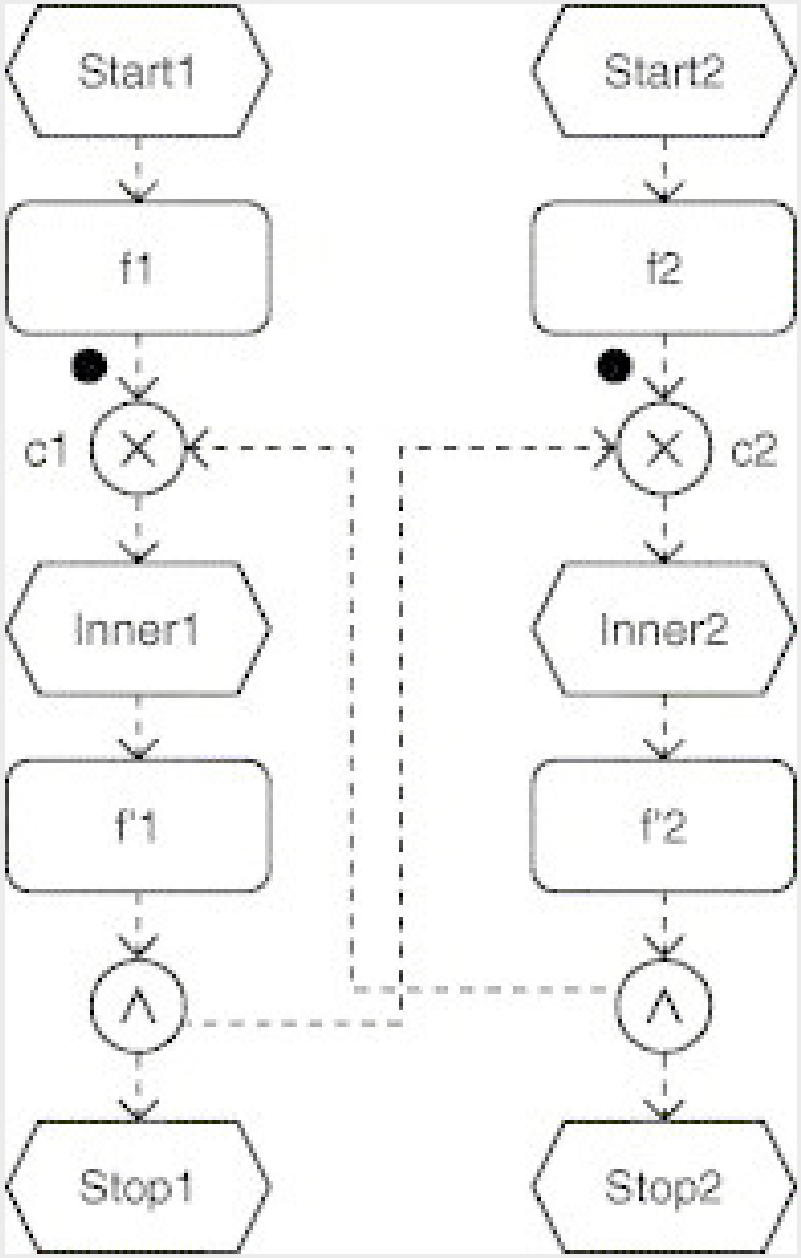
Live Feedback

- Was würden Sie hier tun?

Unlösbares Problem I

- Seit der Definition von EPKs gab es Diskussionen über deren Semantik. [Kin06]
- OR-Join und XOR-Join haben eine nichtlokale informale Semantik.
- Das Problem besteht nicht nur in EPKs sondern auch in anderen Notationen.
- Der XOR-Join in der folgenden Abbildung darf die Prozessmappe auf dem einen Eingang nicht weiterleiten, wenn auf dem anderen noch eine kommen kann (Nichtlokalität).
- Aus Symmetriegründen dürfte entweder keiner von c1 und c2 weiterleiten oder beide.
- Aber in beiden Fällen widerspricht das der informalen Semantik.
- In der ARIS-Simulation leiten beide weiter, was zu einer beliebig großen Zahl von Stop-Ereignissen bei nur einem Startereignis führt.

Unlösbares Problem



[Kin06]

Yet Another Workflow Language

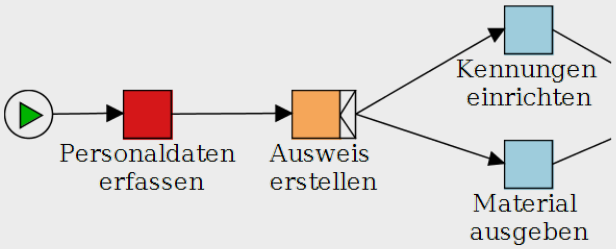
Einführung

YAWL Eigenschaften

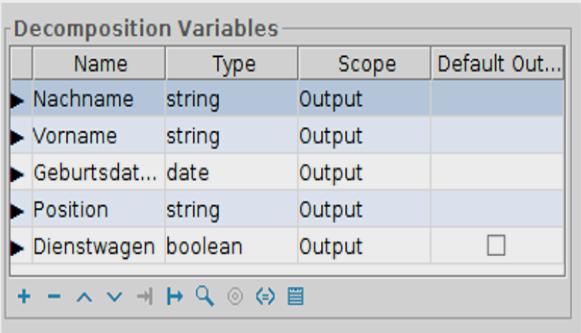
- YAWL wird seit 2002 entwickelt. [HA10]
- YAWL basiert auf Petrinetzen und auf Workflow Patterns [RAH16]
- Petrinetze können die meisten Workflow Patterns darstellen, es mangelt jedoch an Multiple instance patterns, cancellation patterns und dem generalised OR-join.
- YAWL wurde daher um diese Konstrukte erweitert.
- YAWL ist sehr ausdrucksstark, hat aber deutlich weniger Konstrukte als BPMN. [HA10]
- YAWL-Tasks können auf menschliche Teilnehmer, Web-Services, externe Anwendungen oder Java-Klassen abgebildet werden.
- XES Schnittstelle für Process Mining Tools.

Drei Aspekte für das YAWL-WMS

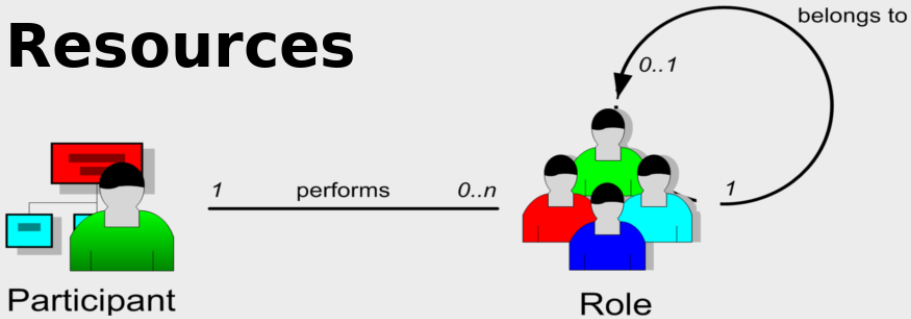
Control Flow



Data



Resources



YAWL

Web App

Yet Another Workflow Language

Anwendungsbeispiel

Order Fulfilment I

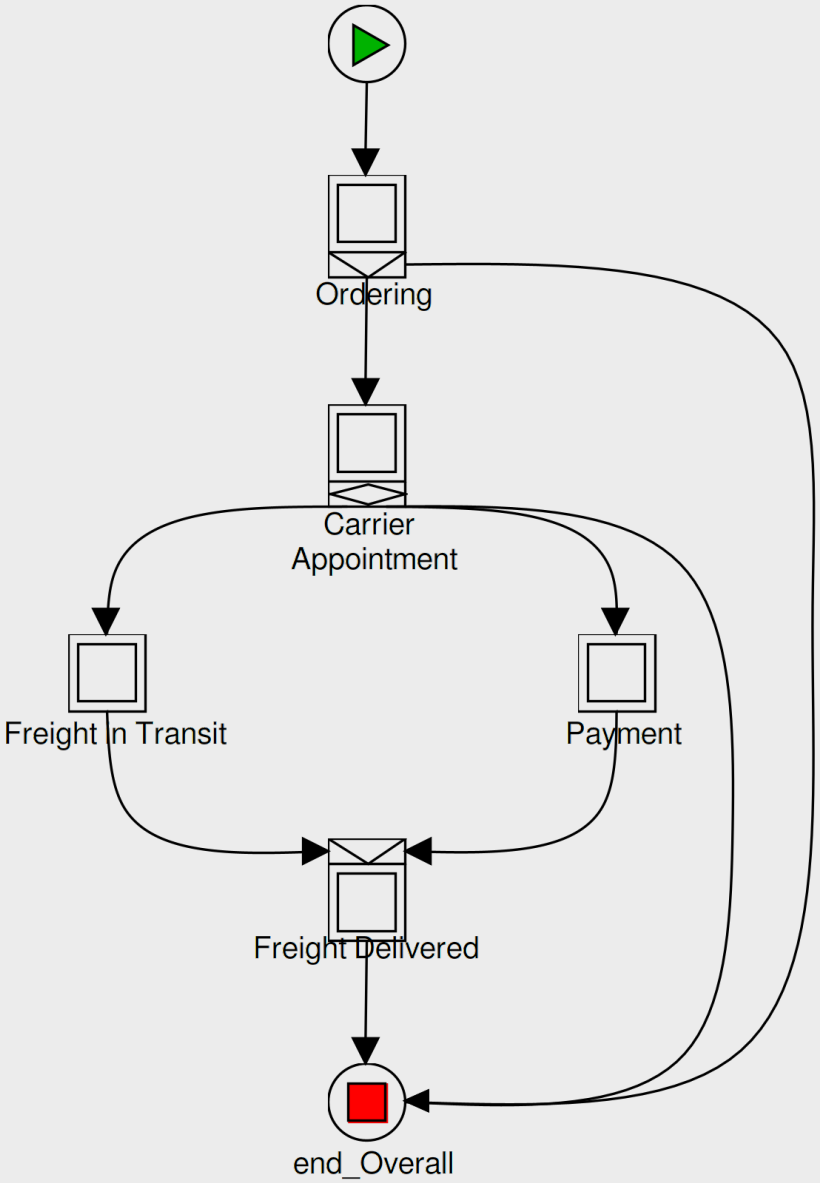
Genko Oil Anwendungsbeispiel aus der Standard YAWL Distribution [LRCH09]

- Genko Oil
- Sitz in Hamburg am Hafen
- 27 Mitarbeiter
  - CEO: Don Vito Corleone
- Produkte von Genko Oil
  - Olivenöl
  - Sonnenblumenöl
  - Pflanzenöl
- Der Auftragsabwicklungsprozess (s. Abb.) hat an der Oberfläche eine recht einfache Struktur.

Order Fulfilment II

- Damit das Beispiel handhabbar bleibt, wurden einige vereinfachende Annahmen gemacht: z. B. führt ein Auftrag nicht zu mehr als einer Lieferung.
- Auf der anderen Seite ist die Zusammenfassung mehrerer Bestellungen zu einer Lieferung möglich.
- Nachdem der Auftrag aufgenommen wurde und das Frachtunternehmen bestimmt wurde, geschehen Transport und Bezahlung parallel.
- Der letzte Subprozess "Freight Delivered" beschäftigt sich mit Schadensersatz etc.

Order Fulfilment

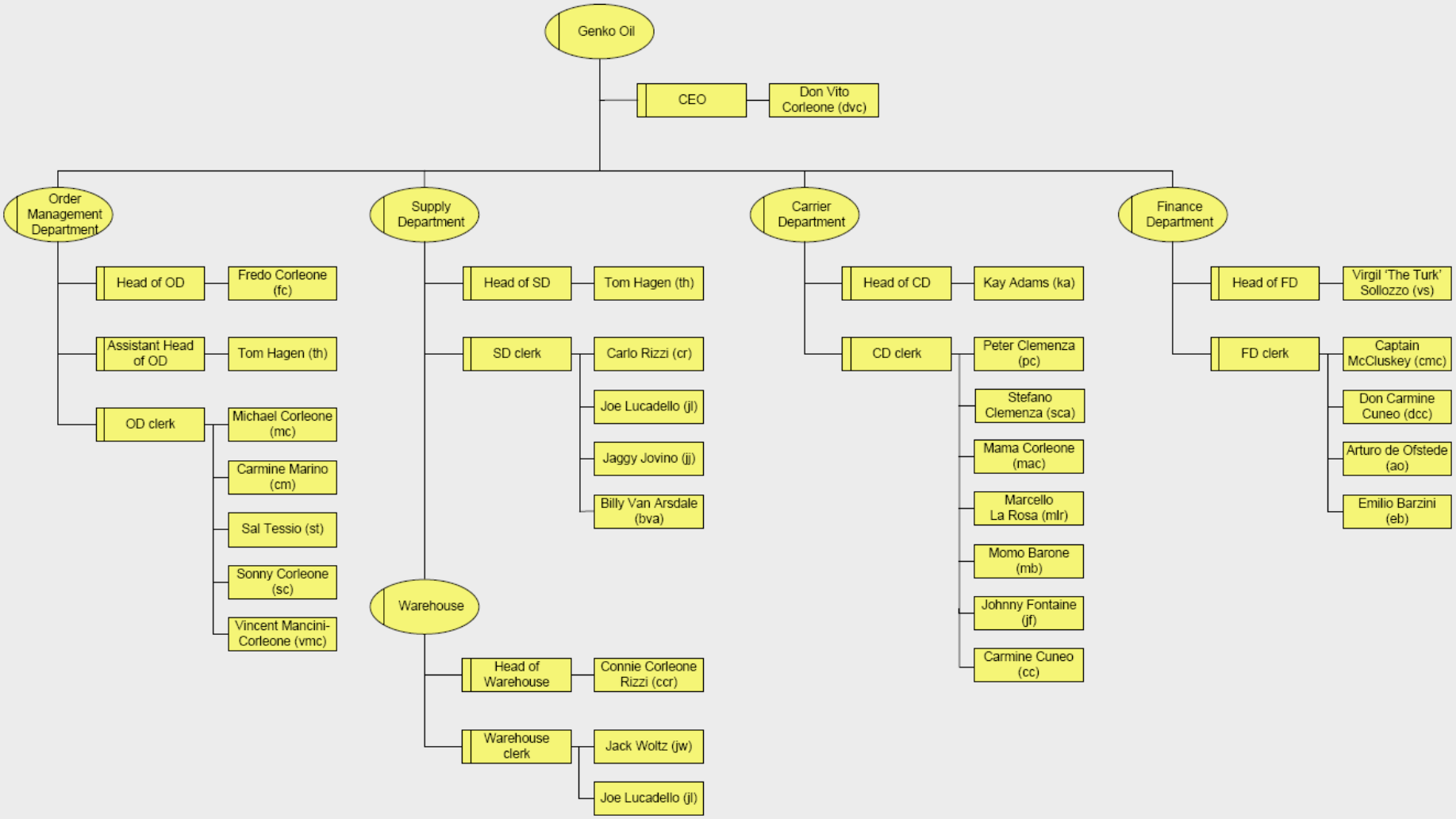


[LRCH09]

Organigramm I

- Don Vito Corleone ist Workflow Admin (eher untypisch in der Praxis).
- Die Führungsrollen der jeweiligen OrgBereiche haben die Möglichkeit, Aufgaben an Mitarbeiter weiterzudelegieren.
- Der Prozess traversiert die gesamte Organisation.

Organigramm

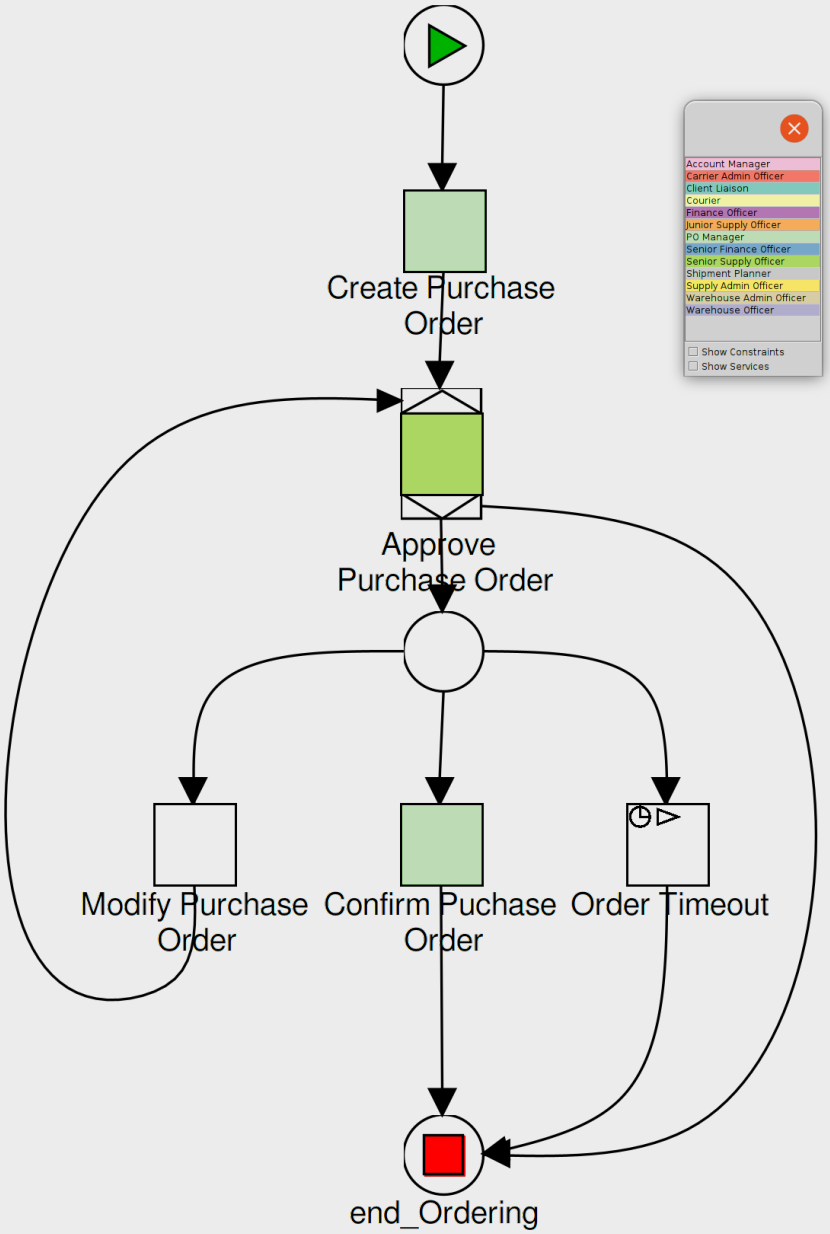


[LRCH09]

Ordering I

- Färbung der Tasks in YAWL nach Ressourcen in [AHH21].
- Zunächst werden vom Order Management Department mit "Create Purchase Order" die Grunddaten ausgefüllt.
- Die Formulardaten können in einer xml-Datei gespeichert und wieder eingelesen werden.
- Das Supply Department setzt dann bei "Approve Purchase Order" ein Häkchen. Achtung: wenn dies nicht geschieht, ist der Case sofort zu Ende.
- Danach geht es für "Confirm Purchase Order" zurück an das Order Management Department.
- Der Order Timeout ist auf 3 Tage eingestellt.
- Zu beachten: die Condition macht die drei nachfolgenden Tasks "feuerbar".

Ordering



[LRCH09]

Fragen

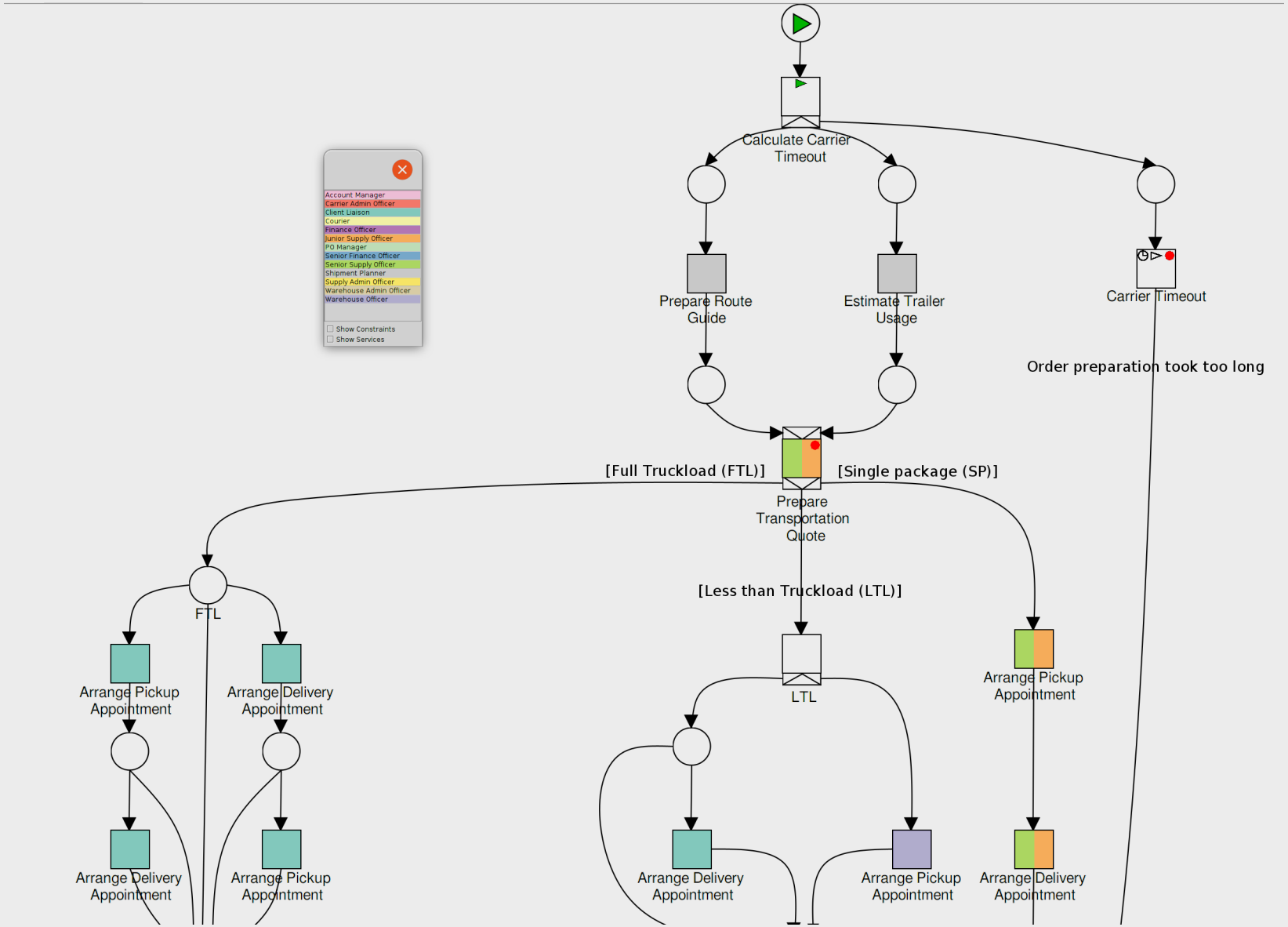
Live Feedback

- Welche Pattern kommen in dem Ordering Sub-Prozess vor?

Carrier Appointment I

- "Calculate Carrier Timeout" berechnet in einem Codelet die Vorbereitungszeit in Abhängigkeit der Anzahl der zu liefernden Waren.
- Zu beachten: gegenseitige Cancellation Sets von "Carrier Timeout" und "Prepare Transportation Quote".
- Vom Supply Department wird "Prepare Transportation Quote" ausgeführt.
- In den Fällen "Truck-Load" und "Less than Truck-Load" werden "Arrange Delivery Appointment" und Arrange Pickup Appointment" aktiviert.

Carrier Appointment

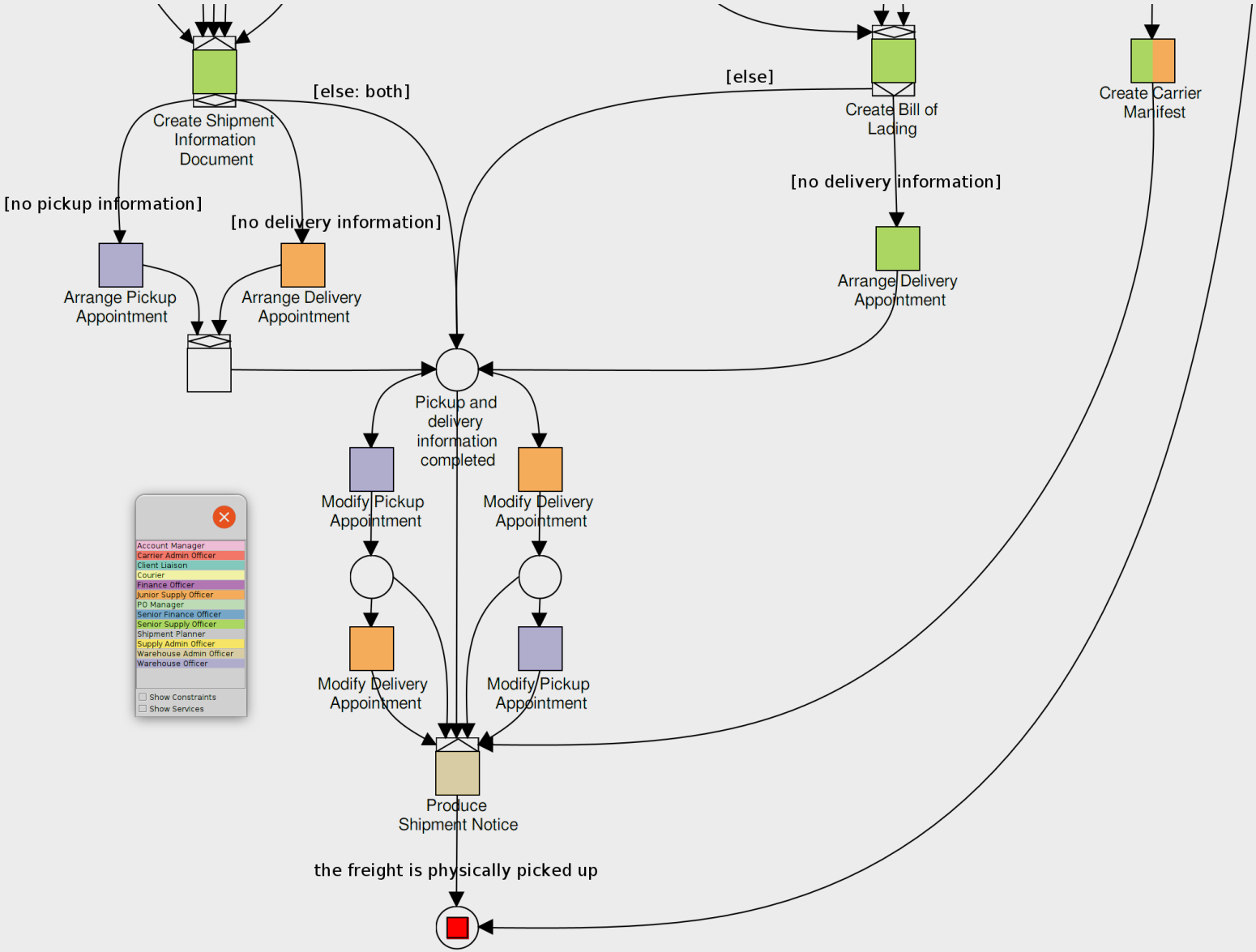


[LRCH09]

Carrier Appointment I

- Beachte: Im Fall "Less than Truck-Load" hat "Create Bill of Lading" einen OR-Join.
- Man beachte die Konstruktion der Modifikationen vor "Produce Shipment Notice"

Carrier Appointment

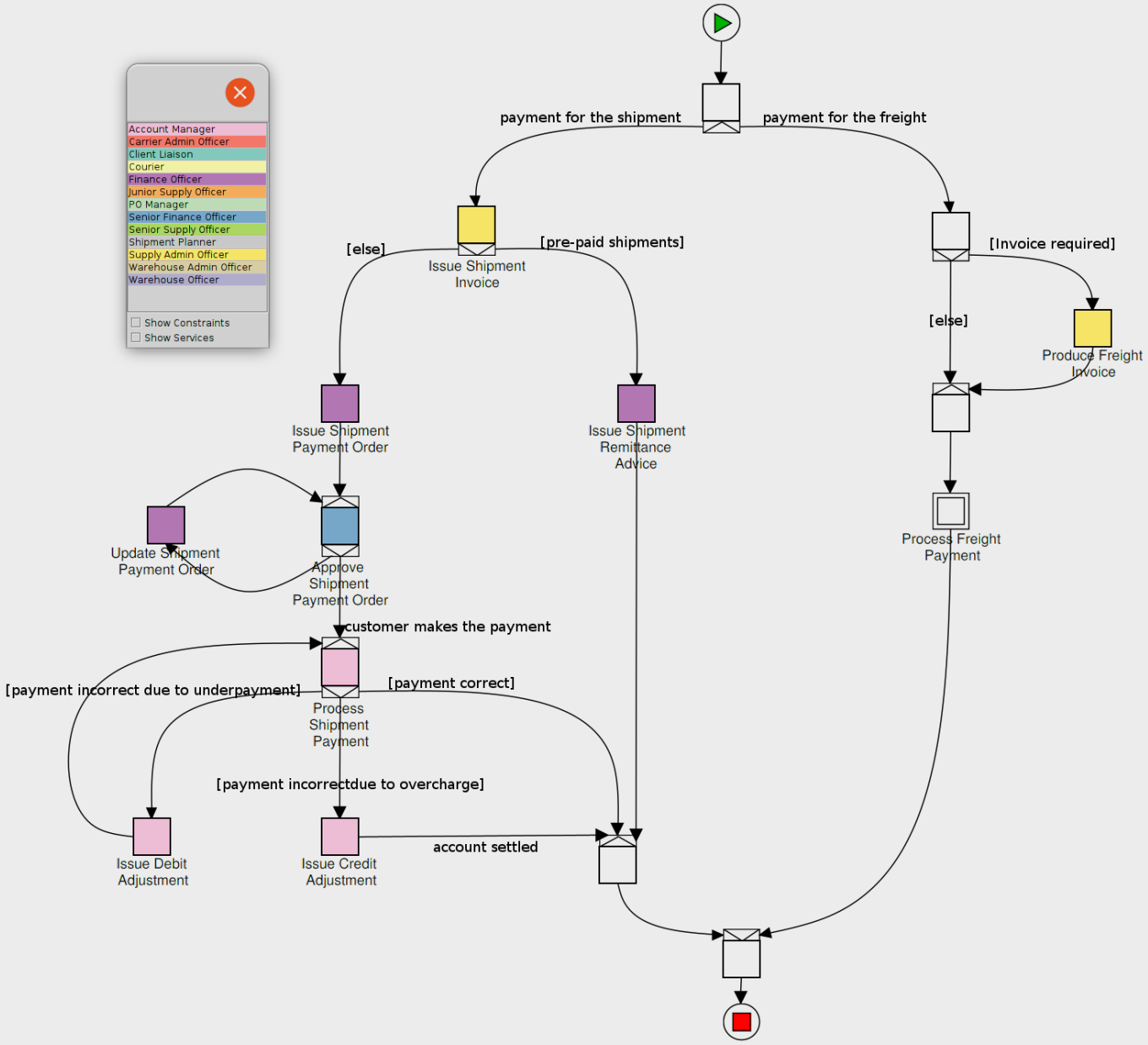


[LRCH09]

Payment I

- Jetzt können "Payment" und "Freight in Transit" parallel laufen.
- Wir schauen uns zunächst "Payment" an.
- Wenn bei "Process Shipment Payment" das Feld "balance" auf null gesetzt wird, so kann der Subprozess enden.
- Der Subprozess Freight Payment ist im Beispiel nicht weiter spezifiziert.

Payment

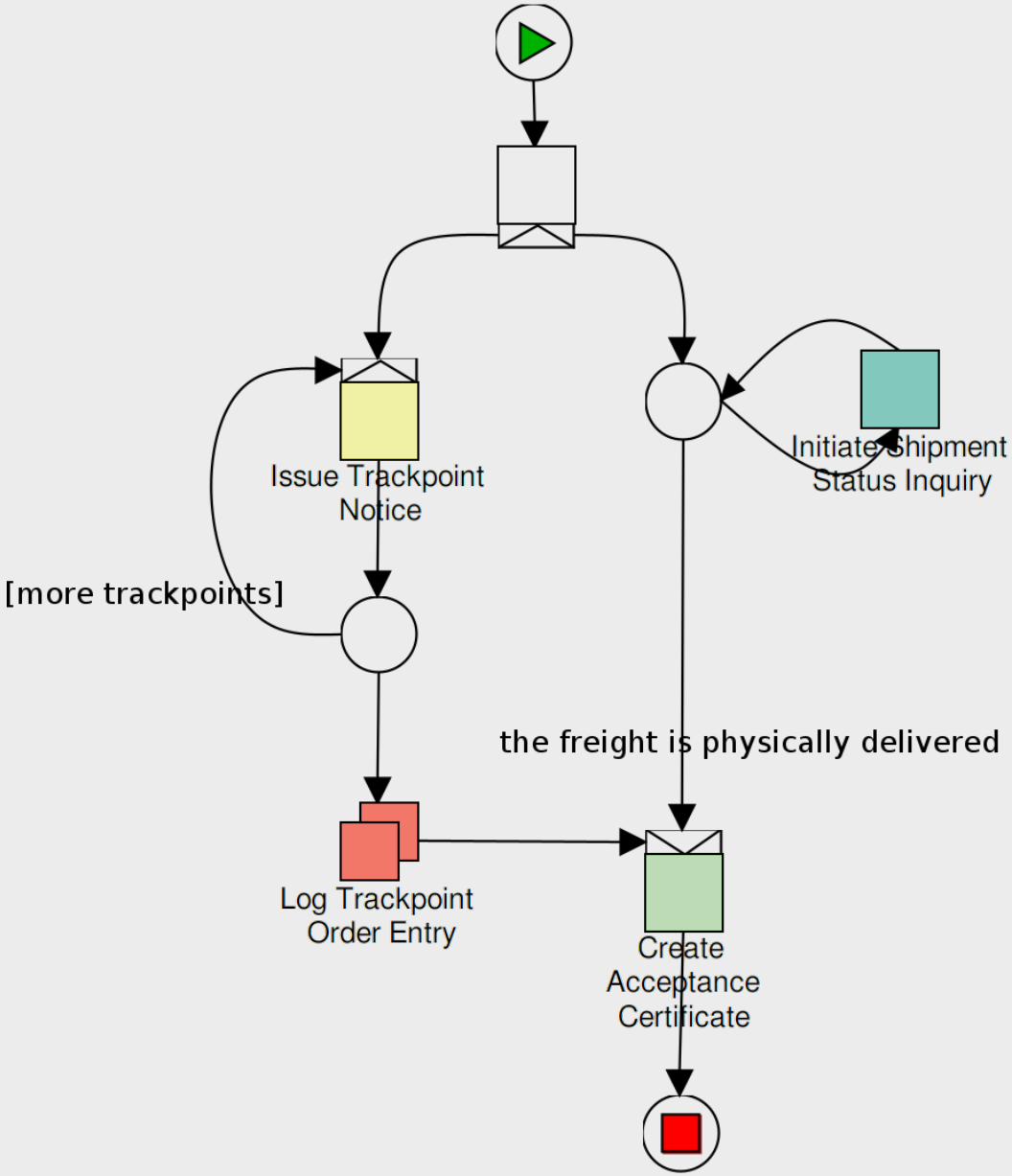


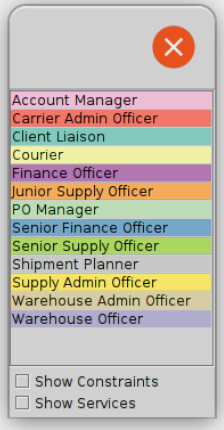
[LRCH09]

Freight in Transit I

- Hier werden Trackpoint Notices gesammelt und dann in einer Multiple Instance Task abgearbeitet.
- Interessant ist auch die Konstruktion bei "Initiate Status Inquiry". Hier besteht die Möglichkeit, so oft wie gewünscht nachzuhaken.

Freight in Transit



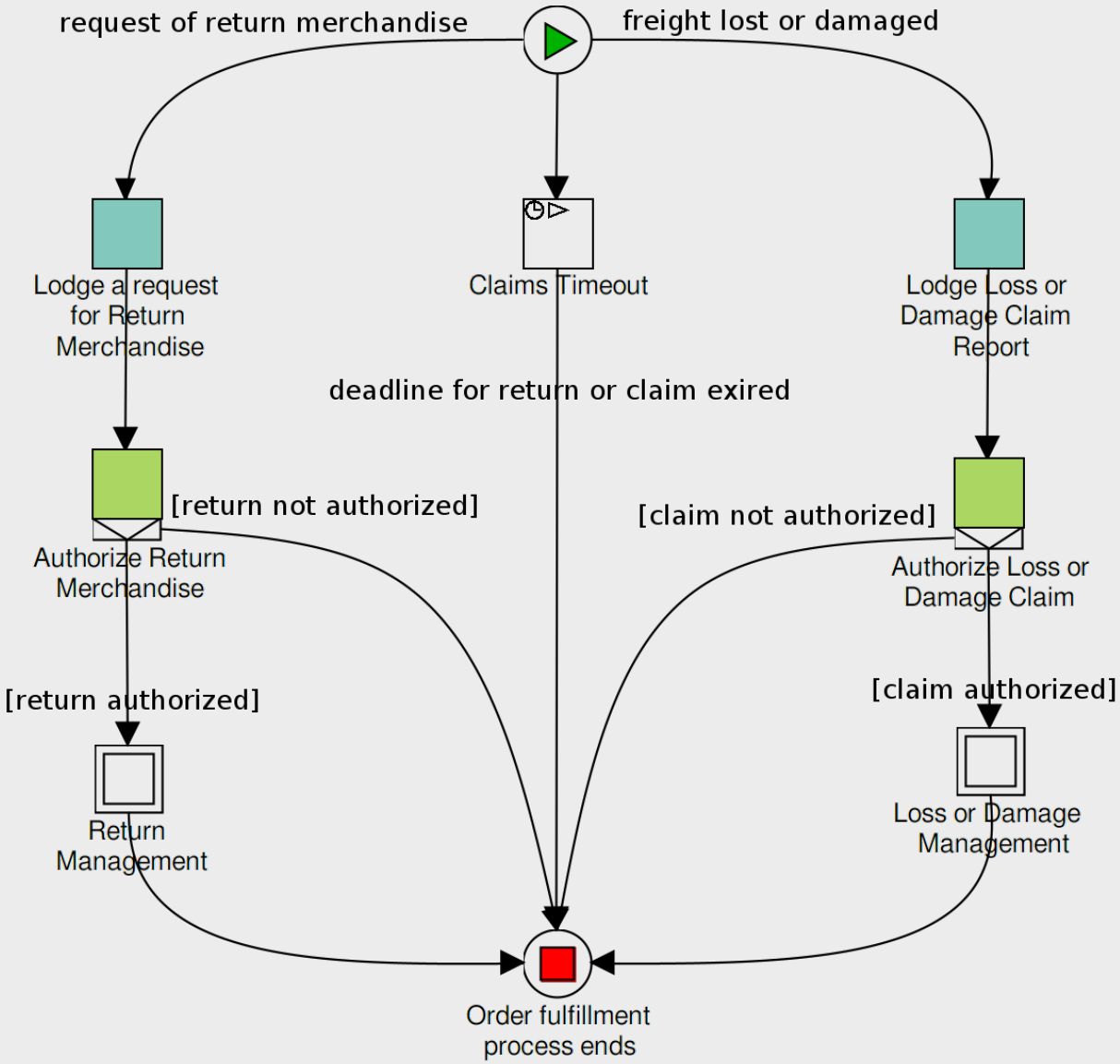


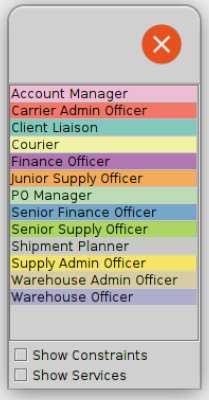
[LRCH09]

Freight Delivered I

- Hier ist nur die oberste Ebene spezifiziert: die beiden Subprozesse sind leer.
- Der Claims Timeout ist über eine Netzvariable vom Typ Input gesetzt.
- Das bedeutet für das Subnetz, dass diese beim Aufruf in "Overall" gesetzt wird.
- In der Tat der Wert aus ShipmentNotice/ClaimsDeadline
- Nach diesem Subprozess endet auch der Hauptprozess.

Freight Delivered





[LRCH09]

Fragen

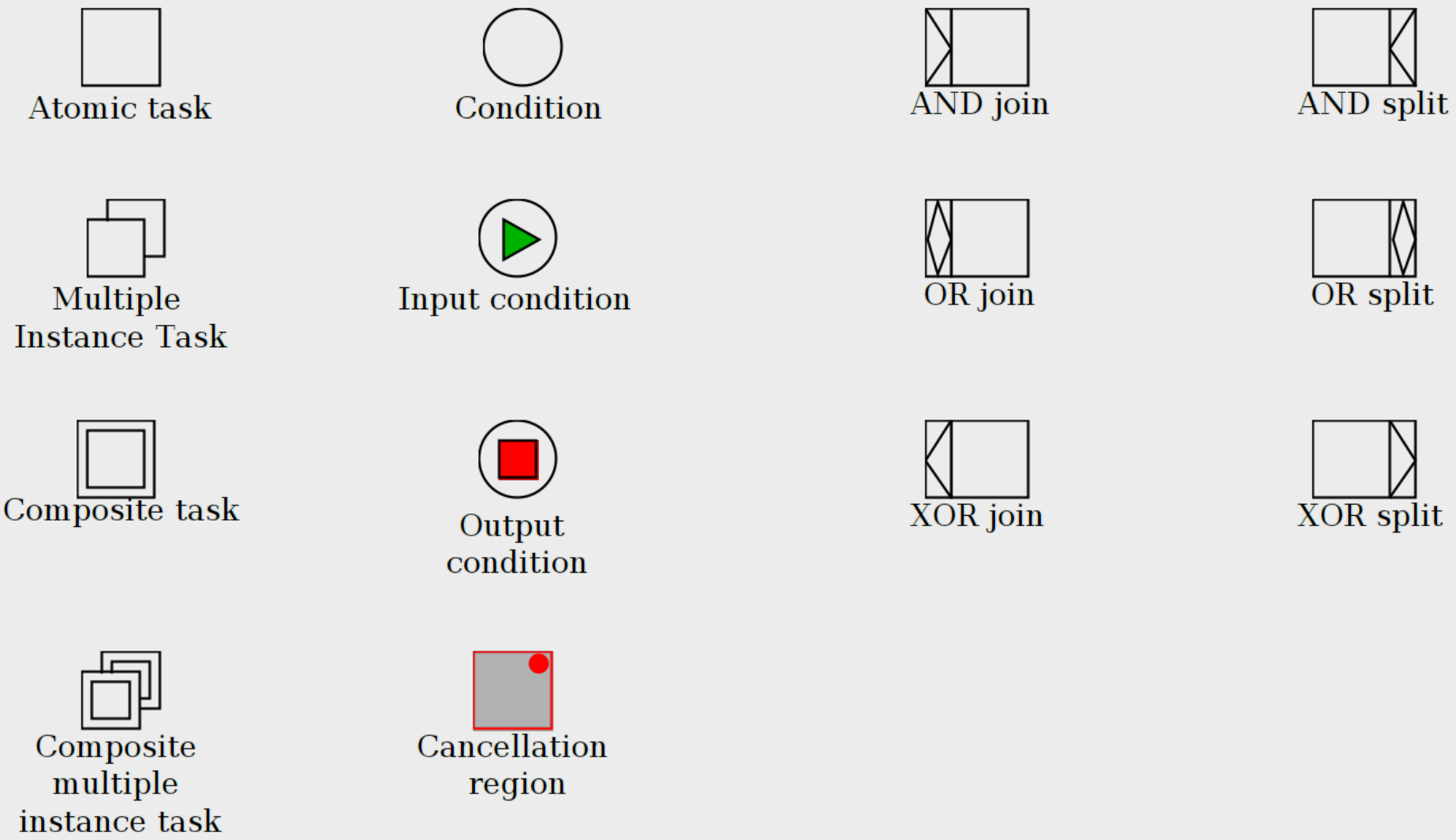
Live Feedback

- Welche Perspektiven werden in den YAWL-Diagrammen von Order Fulfillment gezeigt?

Yet Another Workflow Language

Kontrollfluss Perspektive

YAWL-Kontrollflusssymbole



[HAAR10]

YAWL-Kontrollflusssymbole I

OR-split: die Kontrolle wird einer oder mehreren ausgehenden Kanten übergeben – je nachdem, wie die Auswertungsbedingungen der einzelnen ausgehenden Kanten sind.

XOR-split: es geht genau bei einer der ausgehenden Kanten weiter

Cancellation-region: Eine Gruppe von Conditions und Tasks, die mit einer bestimmten Task t verbunden sind. Wenn t zur Laufzeit zu Ende geführt wird, werden alle Tasks der zugehörigen Cancellation-region gestoppt und alle Marken in Conditions werden entfernt.

AND-join: alle eingehenden Kanten müssen enabled sein.

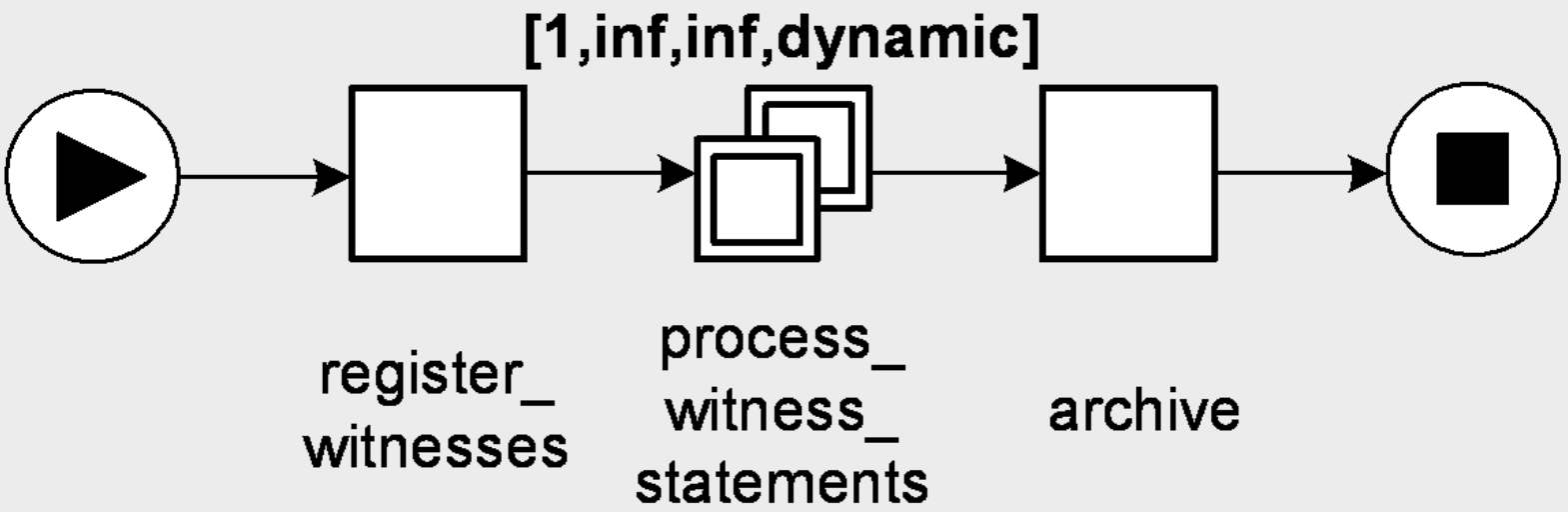
OR-join: entweder sind alle eingehenden Kanten enabled oder es ist mindestens eine enabled und es ist ausgeschlossen, dass irgend eine Kante, die noch nicht enabled war, in Zukunft enabled wird. (non-local semantics[HAAR10] pp 103-119).

YAWL-Kontrollflusssymbole II

XOR-join: eine der beiden eingehenden Kanten enabled

AND-split: es geht bei allen ausgehenden Kanten weiter

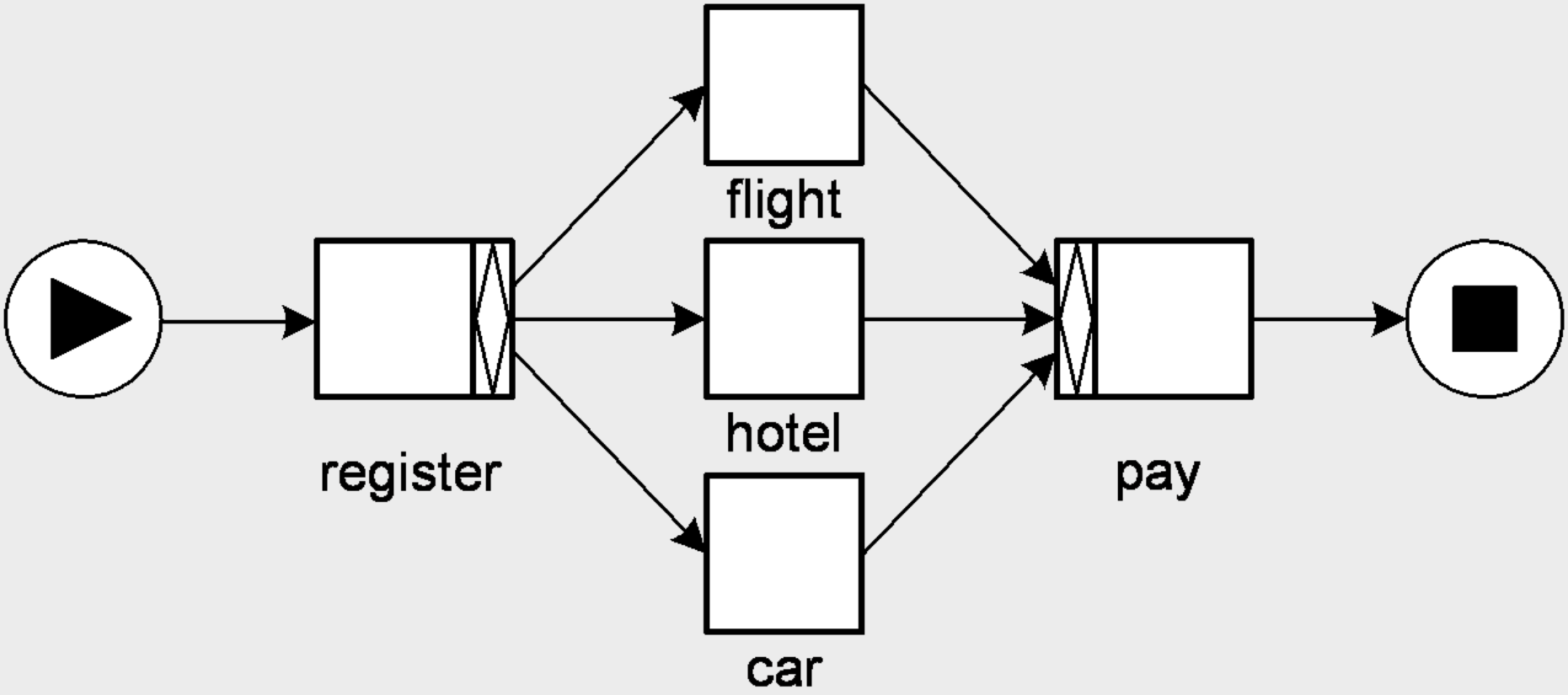
Multiple Instances



An arbitrary number of witnesses can be processed, furthermore, more witnesses can be incorporated after processgin has started but before archiving (Pattern 15).

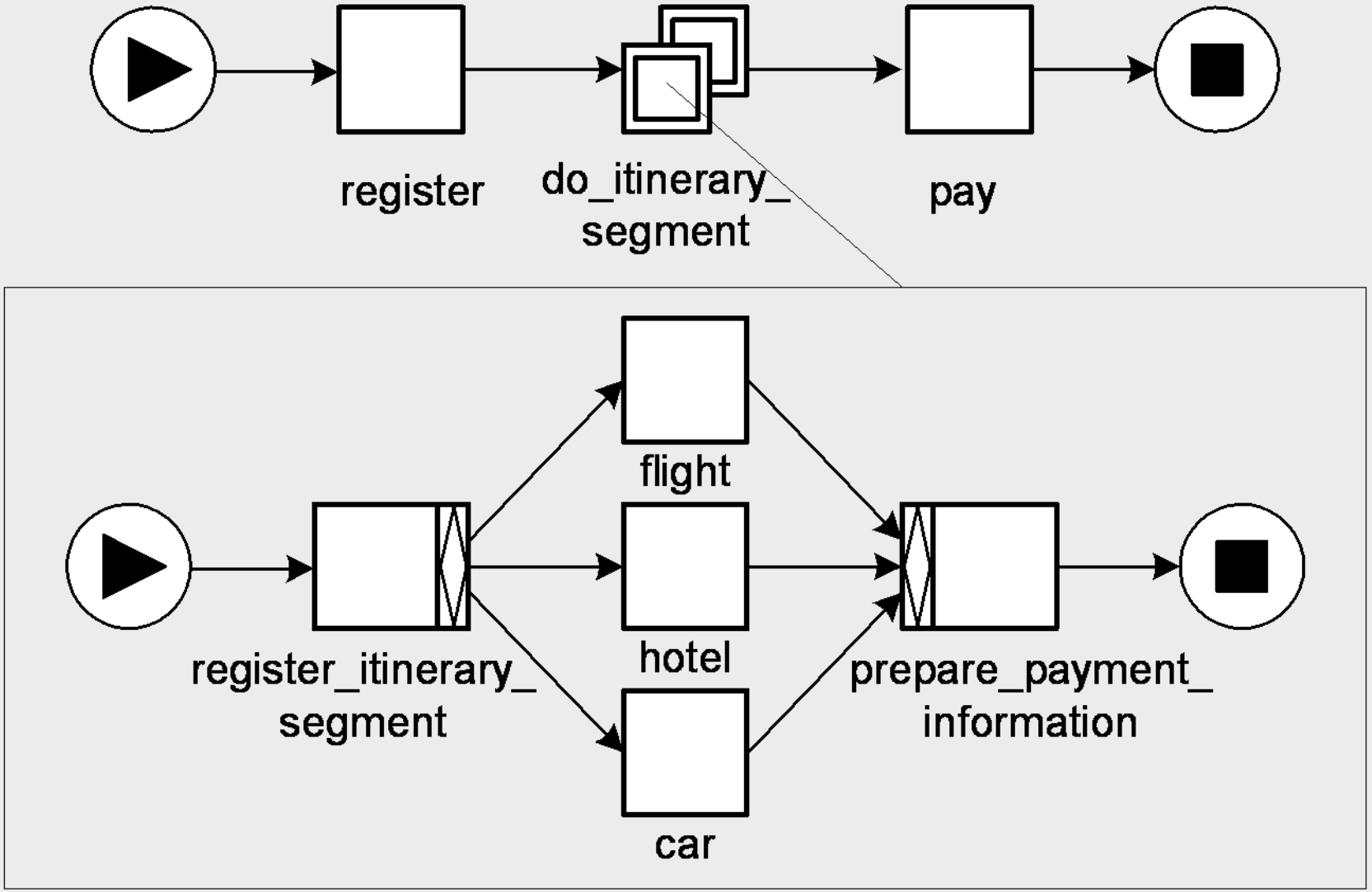
[AAHR09]

Beispiel OR-Split und Join



[AAHR09]

Beispiel Komplexere Buchung



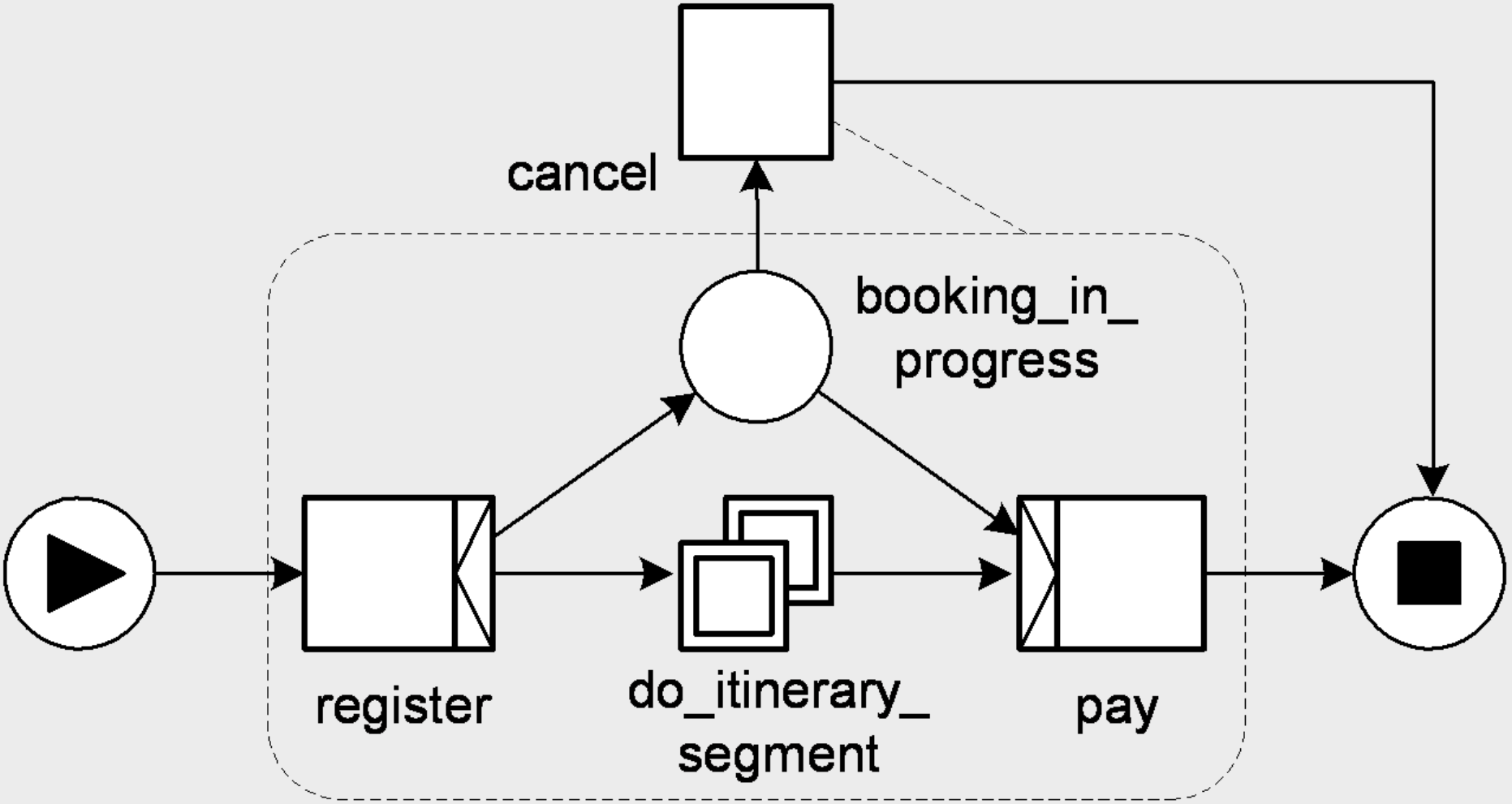
[AAHR09]

Fragen

Live Feedback

- Wie viele Kombinationen bzw. Ausführungsreihenfolgen gibt es im Beispiel Komplexere Buchung, wenn wir ein Segment haben?

Cancellation Example



[AAHR09]

Cancellation Examples I

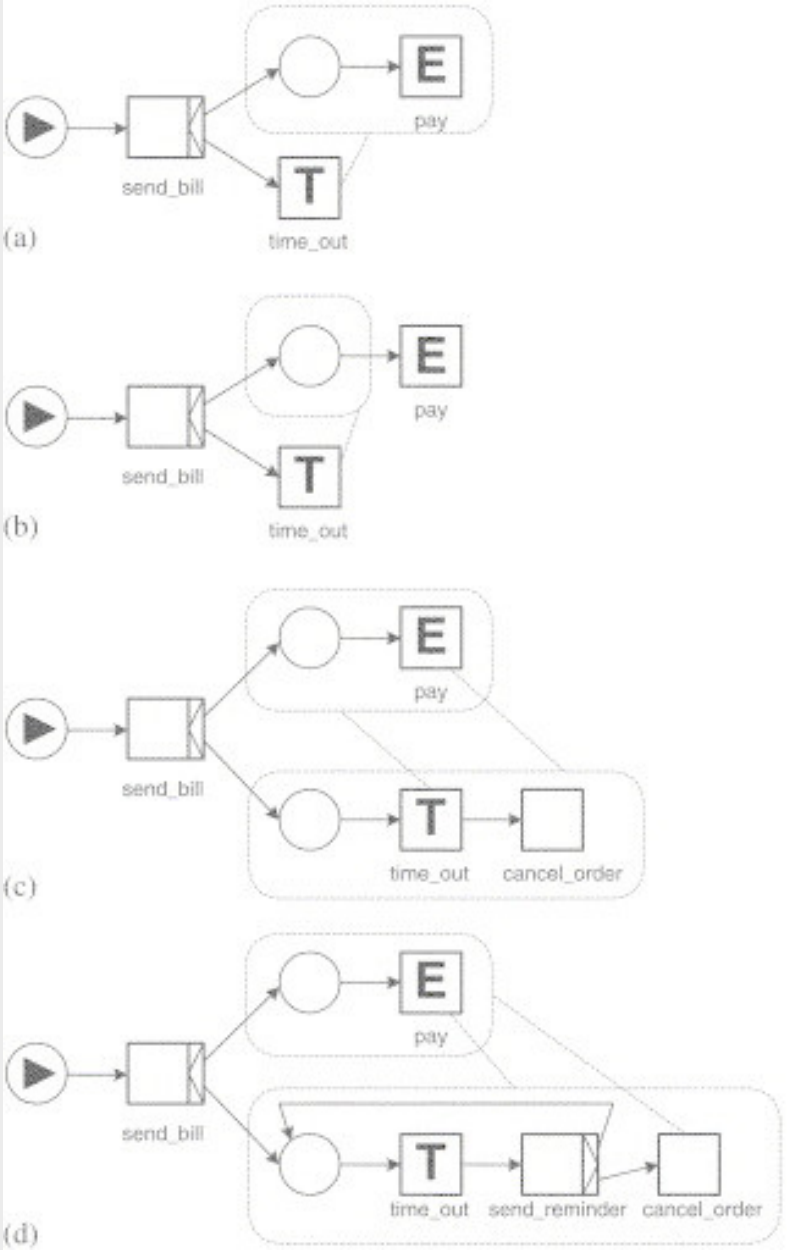
a Die Timeout Task T bricht den ganzen Payment Subprozess ab.

b Der Payment Subprozess wird nur abgebrochen, wenn er noch nicht gestartet ist.

c Wie unter a, nur dass jetzt der Payment Subprozess den Timer Subprozess abbricht.

d Hier werden noch einige Erinnerungen versandt, bevor der Payment Subprozess abgebrochen wird.

Cancellation Examples



[AH05]

Yet Another Workflow Language

XPATH

- XPATH ist eine Sprache, mit der Teile eines XML-Dokuments bezeichnet werden können. [CD99].
- Wichtigstes Element sind Location Paths, die aus Location Steps bestehen.
- Ein Location Step besteht aus drei Teilen:

  Axis beschreibt die Relation im Baum zwischen den selektierten Knoten und dem Kontext Knoten.

  Node test Knotentyp und den Knoten-Namen der selektierten Knoten.

  Predicates null oder mehr Prädikate, die die Knotenmenge weiter verfeinern.

Um XPATH auszuwerten, nutzen wir BaseX.

Grundoperatoren am Beispiel I

```
<?xml version = "1.0"?>
<class>
   <student rollno = "393">
      <firstname>Dinkar</firstname>
      <lastname>Kad</lastname>
      <nickname>Dinkar</nickname>
      <marks>85</marks>
   </student>
   <student rollno = "493">
      <firstname>Vaneet</firstname>
      <lastname>Gupta</lastname>
      <nickname>Vinni</nickname>
      <marks>95</marks>
   </student>
   <student rollno = "593">
      <firstname>Jasvir</firstname>
      <lastname>Singh</lastname>
      <nickname>Jazz</nickname>
      <marks>90</marks>
```

Grundoperatoren am Beispiel II

```
    </student>
</class>
```

students.xml aus Quelle https://www.tutorialspoint.com

Der folgende Location path besteht aus drei Steps:

```
/child::class/child::student[position()=2]
```

child ist der Name der Axis, class ist ein Node test und in den eckigen Klammern steht ein Prädikat. In dem obigen Beispiel selektiert er den zweiten Studenten.

Es gibt die folgenden Achsen, jeweils mit Beispiel aus students.xml. Alles bezieht sich jeweils auf den oder die Kontext-Knoten, wo man gerade steht.

child Die Kinder. /descendant::student/child::*

descendant Alle Nachfahren. /child::class/descendant::*

Grundoperatoren am Beispiel III

parent Vaterknoten, falls er existiert.
`/descendant::nickname[text() = "Jazz"]/parent::*`

ancestor Alle Vorfahren. `/descendant::nickname[text() = "Jazz"]/ancestor::*`

following-sibling Die folgenden Geschwister.
`/descendant::nickname[text() = "Dinkar"]/../following-sibling::*`

preceding-sibling Die vorhergehenden Geschwister.
`/descendant::nickname[text() = "Jazz"]/../preceding-sibling::*`

following Alle im Dokument folgenden Knoten außer Nachfahren.
`/descendant::nickname[text() = "Dinkar"]/../following::*`

Grundoperatoren am Beispiel IV

preceding Alle im Dokument vorhergehenden Knoten außer Vorfahren. `/descendant::nickname[text() = "Jazz"]/../preceding::*`

attribute Die Attribute. `/descendant::nickname[text() = "Jazz"]/../attribute::*`

namespace Die Namespace-Knoten. Ist in Xpath 1.0 nicht immer implementiert.

self Der Knoten selbst. `/descendant::nickname[text() = "Jazz"]/../self::*`

descendant-or-self Der Knoten selbst und seine Nachfahren. `/child::class/descendant-or-self::*`

ancestor-or-self Der Knoten selbst und seine Vorfahren. `/descendant::nickname[text() = "Jazz"]/ancestor-or-self::*`

Grundoperatoren am Beispiel V

Jede Achse hat einen "principal node type". Wenn eine Achse Elemente enhält, ist es element. Sonst der Typ von Knoten, den die Achse enthalten kann. Wir können jezt folgende Node-Tests ausführen:

- `*` Wahr für alle Knoten des "principal node type" `//child::element()` und `//child::*` liefern im obigen Beispiel dasselbe Ergebnis.
- `node()` Alle Knoten. `//child::node()`
- `text()` Alle Text Knoten. `//child::text()`

Es gibt viele Abkürzungen, die wir oben teilweise schon verwendet haben. Einige der wichtigsten:

- `nodename` `child::nodename`
- `//` `/descendant-or-self::node()/`
- `.` `self::node()`
- `..` `parent::node()`
- `@` `attribute::`

Weitere Beispiele I

Wir folgen [w3s13a].

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
<book>
  <title lang="eng">Harry Potter</title>
  <price>29.99</price>
</book>
<book>
  <title lang="de">XML Lernen</title>
  <price>39.95</price>
</book>
</bookstore>
```

200_buecher

Der folgende XPATH-Ausdruck

```
//title[@lang='de']
```

Weitere Beispiele II

erzeugt die Ausgabe

```
<title lang="de">XML Lernen</title>
```

Live Feedback

- Wie lautet der XPATH-Ausdruck, der nicht alle deutschen Titel sondern alle deutschen Bücher selektiert?

Syntax I

Der folgende XPATH-Ausdruck

```
/bookstore/book[price>35.00]/title
```

erzeugt die Ausgabe

```
<title lang="de">XML Lernen</title>
```

Der folgende XPATH-Ausdruck

```
//title | //price
```

erzeugt die Ausgabe

```
<title lang="eng">Harry Potter</title>
<price>29.99</price>
<title lang="de">XML Lernen</title>
<price>39.95</price>
```

Was ist der Unterschied zwischen

Syntax II

```
/*
```

und

```
//*
```

in unserem Beispiel?

Der folgende XPATH-Ausdruck

```
//book[1]/price + //book[2]/price
```

erzeugt die Ausgabe

```
69.94
```

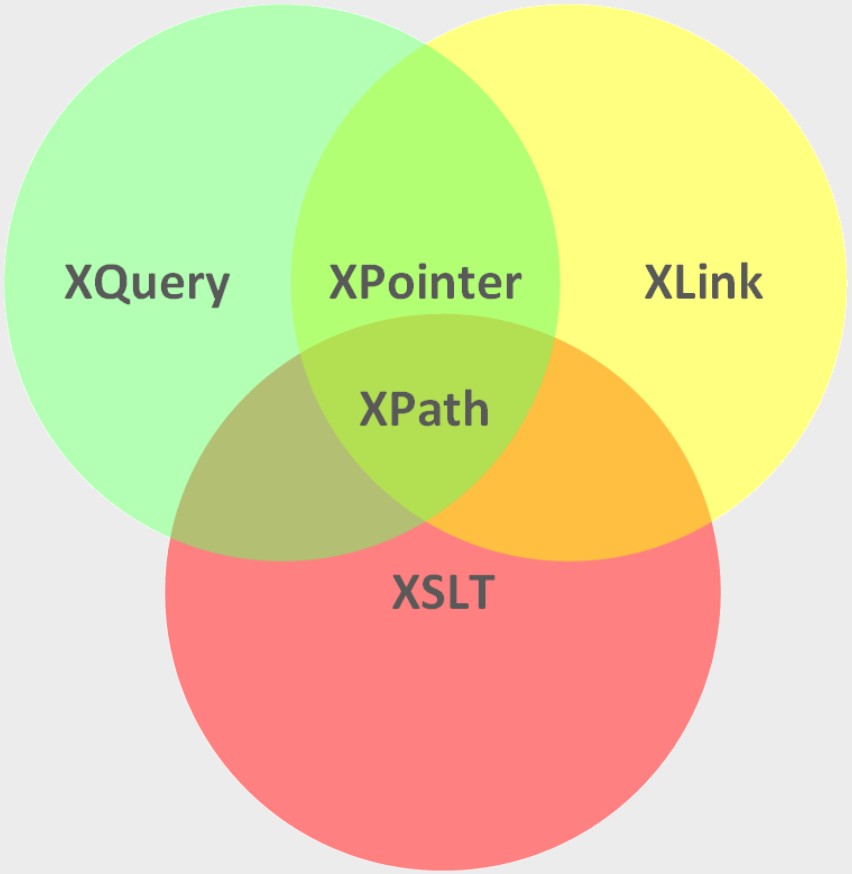
Nutzung von XPATH

XQuery Abfragesprache für XML-Dokumente.

XSLT Sprache zur Transformation von XML-Dokumenten.

XLink Hyperlinks in XML-Dokumenten.

XPointer Teile von XML-Dokumenten referenzieren.



[w3s13a]

Yet Another Workflow Language

XQUERY

- XQUERY ist eine Abfragesprache für XML-Dokumente.
- Um XQUERY auszuwerten, nutzen wir BaseX (im ubuntu repo, auch für win und mac).
- Wir folgen [w3s13b].

Beispieldatei I

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<bookstore>
<book category="COOKING">
  <title lang="en">Everyday Italian</title>
  <author>Giada De Laurentiis</author>
  <year>2005</year>
  <price>30.00</price>
</book>
<book category="CHILDREN">
  <title lang="en">Harry Potter</title>
  <author>J K. Rowling</author>
  <year>2005</year>
  <price>29.99</price>
</book>
<book category="WEB">
  <title lang="en">XQuery Kick Start</title>
  <author>James McGovern</author>
  <author>Per Bothner</author>
  <author>Kurt Cagle</author>
```

Beispieldatei II

```
  <author>James Linn</author>
  <author>Vaidyanathan Nagarajan</author>
  <year>2003</year>
  <price>49.99</price>
</book>
<book category="WEB">
  <title lang="en">Learning XML</title>
  <author>Erik T. Ray</author>
  <year>2003</year>
  <price>39.95</price>
</book>
</bookstore>
```

FLOWR I

Der folgende XPATH-Ausdruck

```
/bookstore/book[price>30]/title
```

erzeugt die Ausgabe

```
<title lang="en">XQuery Kick Start</title>
<title lang="en">Learning XML</title>
```

In XQUERY sind zusätzlich FLOWR-Ausdrücke möglich. (FOR, LET, ORDER BY, WHERE, RETURN) Dasselbe Resultat bekommen wir mit

```
for $x in /bookstore/book
where $x/price>30
return $x/title
```

Aber jetzt können wir die Ausgabe ordnen:

FLOWR II

```
for $x in doc("books.xml")/bookstore/book
where $x/price>30
order by $x/title
return $x/title
```

Oder auch HTML hinzufügen – geschweifte Klammern werden immer dann gebraucht, wenn innerhalb von XML-Markup XQUERY-Ausdrücke ausgewertet werden sollen.

```
<ul>
{ for $x in /bookstore/book/title/text()
order by $x
return <li>{$x}</li>
}
</ul>
```

erzeugt die Ausgabe

FLOWR III

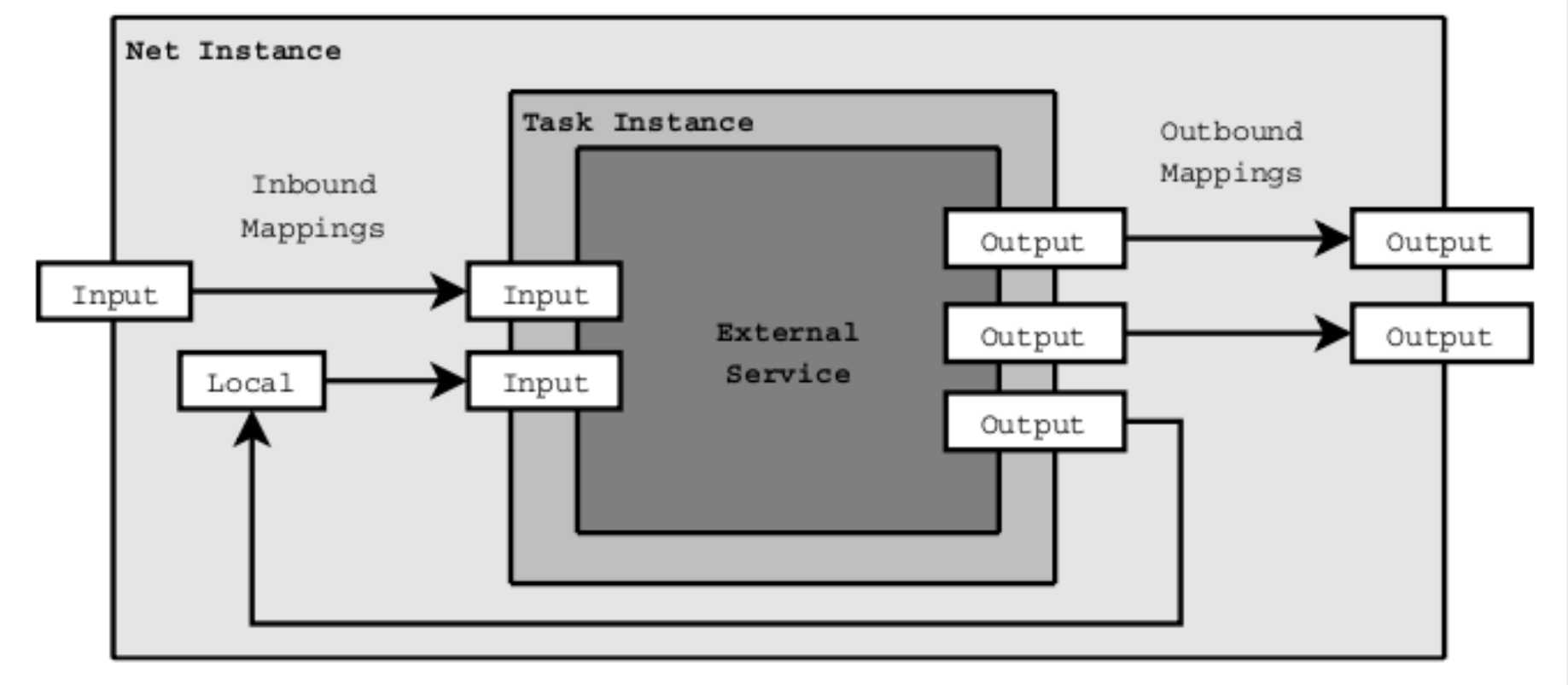
```
<ul>
  <li>Everyday Italian</li>
  <li>Harry Potter</li>
  <li>Learning XML</li>
  <li>XQuery Kick Start</li>
</ul>
```

Yet Another Workflow Language

Daten Perspektive

- In YAWL unterscheidet man zwischen Netzvariablen und Taskvariablen (s. Abb. unten).
- Variablen können als Input oder Output dienen oder auch lokal sein.
- YAWL nutzt XML Schema für die Datenspezifikation.
- Im Editor kann man aus einer Reihe von primitiven Datentypen wählen.
- Will man eigene Datentypen definieren, so werden diese in der Spezifikation als Datentyp Definitionen hinterlegt.
- Sinnvoll ist, die Datentyp Definitionen (.xsd) in einer Entwicklungsumgebung zu erstellen und daraus Werte (.xml) zu erzeugen, um Variablen im Netz zu initialisieren, falls das nötig ist.

YAWL Data Transfer



[HA10]

xsd und xml I

Inhalt der .xsd Datei

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:simpleType name="FormOfEmploymentType">
    <xs:restriction base="xs:string">
      <xs:enumeration value="Full Time" />
      <xs:enumeration value="Part Time" />
      <xs:enumeration value="Trainee" />
    </xs:restriction>
  </xs:simpleType>
<xs:element name="FormOfEmployment"
  type="FormOfEmploymentType" />
</xs:schema>
```

generiert die folgende .xml Datei

xsd und xml II

```
<?xml version="1.0" encoding="UTF-8"?>
<FormOfEmployment
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="HireEmployee.xsd">
  Full Time
</FormOfEmployment>
```

Umgekehrt kann man auch die .xml Datei gegen das Schema validieren lassen.

Datenbasiertes Routing I

- Die meisten WMS nutzen eine proprietäre Sprache für die Verarbeitung von Daten.
- YAWL nutzt den Standard XQUERY/XPATH, um Daten zwischen den zwischen den Tasks auszutauschen.
- Wollen wir z. B. auf eine boole'sche Netzvariable namens accepted prüfen, so können wir folgendes Prädikat verwenden:
- /MyWorkflow/accepted/text() = 'true'
- Beim XOR Split werden die Prädikate von oben nach unten ausgewertet und das erste genommen, welches sich zu true auswertet.
- Beim OR Split werden alle Prädikate ausgewertet und alle Pfade mit true aktiviert. Der letzte Pfad ist dabei default.

Multiple Instance Tasks

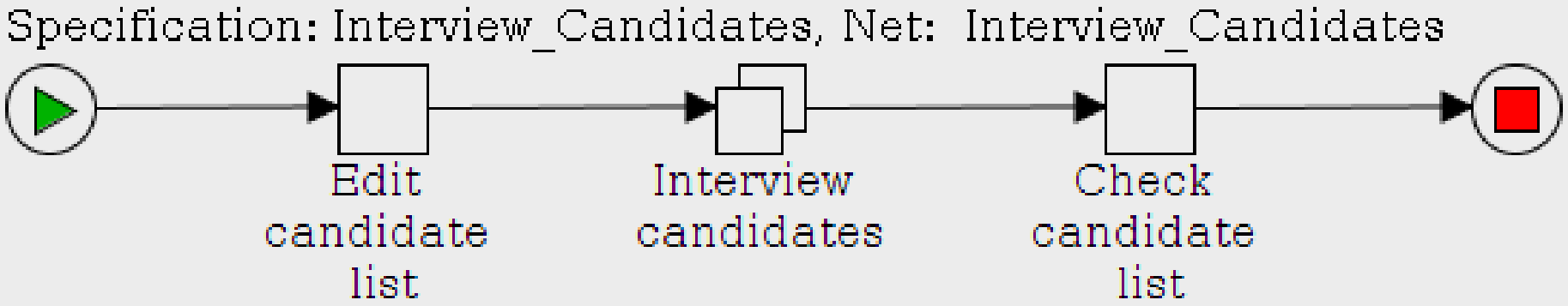
Demo mit YAWL-Editor

- Wir stellen uns einen Personalauswahlprozess vor.
- Zunächst wird eine Liste von vielversprechenden Kandidaten aus den schriftlichen Bewerbungen erstellt.
- Dann soll mit jedem der Kandidaten der Liste ein telefonisches Vorgespräch geführt werden.
- Die Vorgespräche sollen von mehreren Mitarbeitern des Unternehmens durchgeführt werden können.

Nach Tutorial `http://www.yaug.org/content/using-multiple-instance-tasks`

- Die nächste Seite zeigt den einfachen Prozessablauf mit einer MI Task in der Mitte.

Multiple Instance Tasks



[Hen12]

Multiple Instance Tasks

Wir benötigen folgende Datentypen:

```
<xs:complexType name="candidateType">
   <xs:sequence>
      <xs:element name="Name" type="xs:string"/>
      <xs:element name="Phone_number" type="xs:string"/>
      <xs:element name="Comment" type="xs:string"/>
   </xs:sequence>
</xs:complexType>
<xs:complexType name="candidatesType">
   <xs:sequence>
      <xs:element name="Candidate" type="candidateType"
           minOccurs="1" maxOccurs="unbounded"/>
   </xs:sequence>
</xs:complexType>
```

Multiple Instance Tasks

- Wir deklarieren eine Netzvariable candidates:candidatesType und initialisieren diese.
- Wir nutzen diese Variable als input/output in "Edit candidate list" und als input in "Check candidate list".
- Außerdem ziehen wir sie in die Task "Interview candidates".
- Wir klicken dann dort auf den Button "mark as MI" (s. nächste Folie).

Multiple Instance Tasks

Data Variables for Decomposition Interview_candidates [Task: Interview_candidates]

Net Variables

Name	Type	Scope	Initial Value
candidates	candidatesType	Local	<Candidate> <Name>...

Decomposition Variables

Name	Type	Scope	Default Output Value
candidates_Item	candidateType	InputOutput	###

Cancel | Apply | OK

[Hen12]

Multiple Instance Tasks

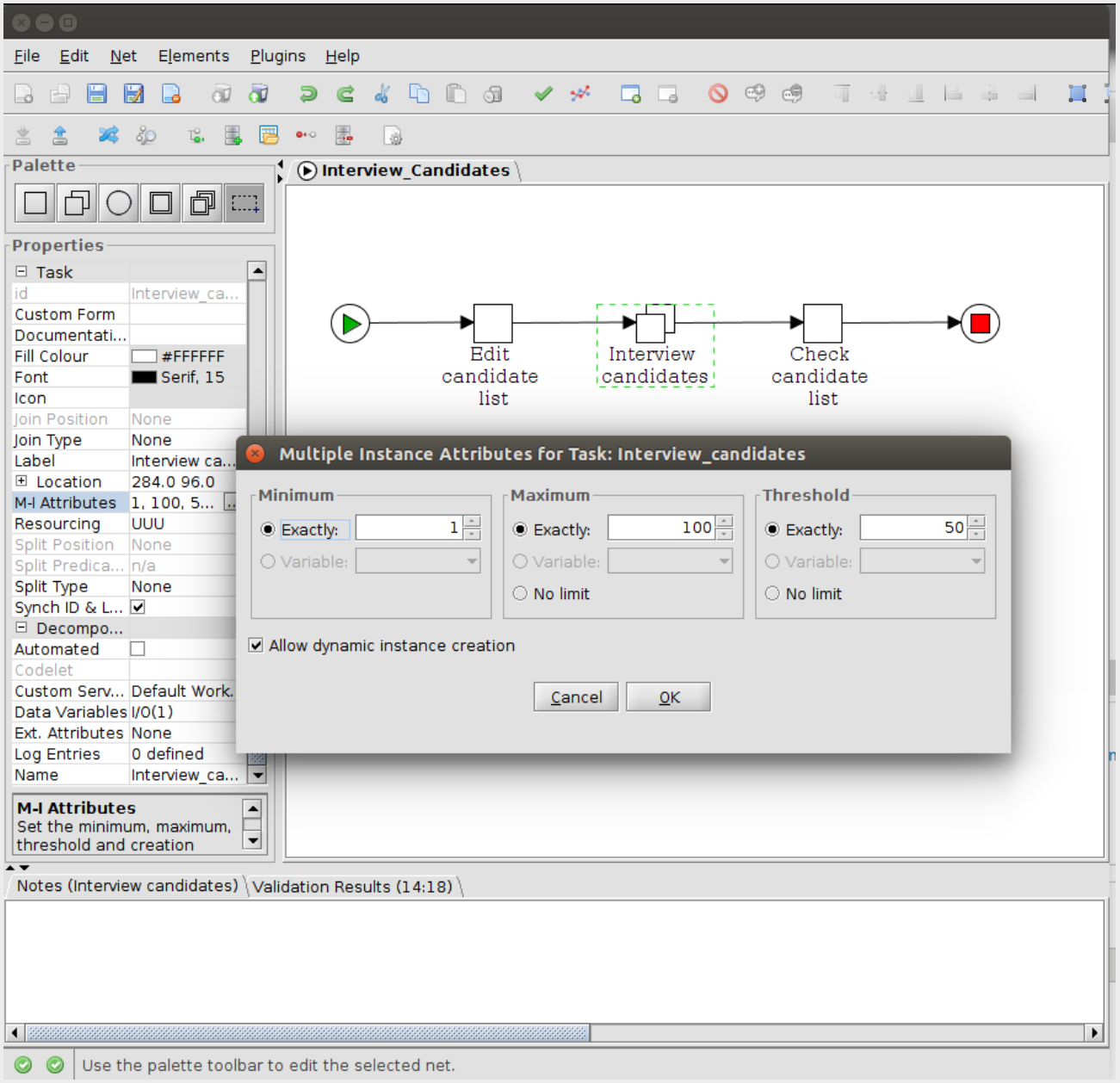
Das vom Editor generierte Input Mapping ist dieses (splitter query):

```
for $s in /candidates/*
return <candidates_Item>{$s/*}</candidates_Item>
```

Das vom Editor generierte Output Mapping ist dieses (joining query):

```
for $j in /Interview_candidates/Candidate
return $j
```

Multiple Instance Attribute



[Hen12]

Multiple Instance Attribute I

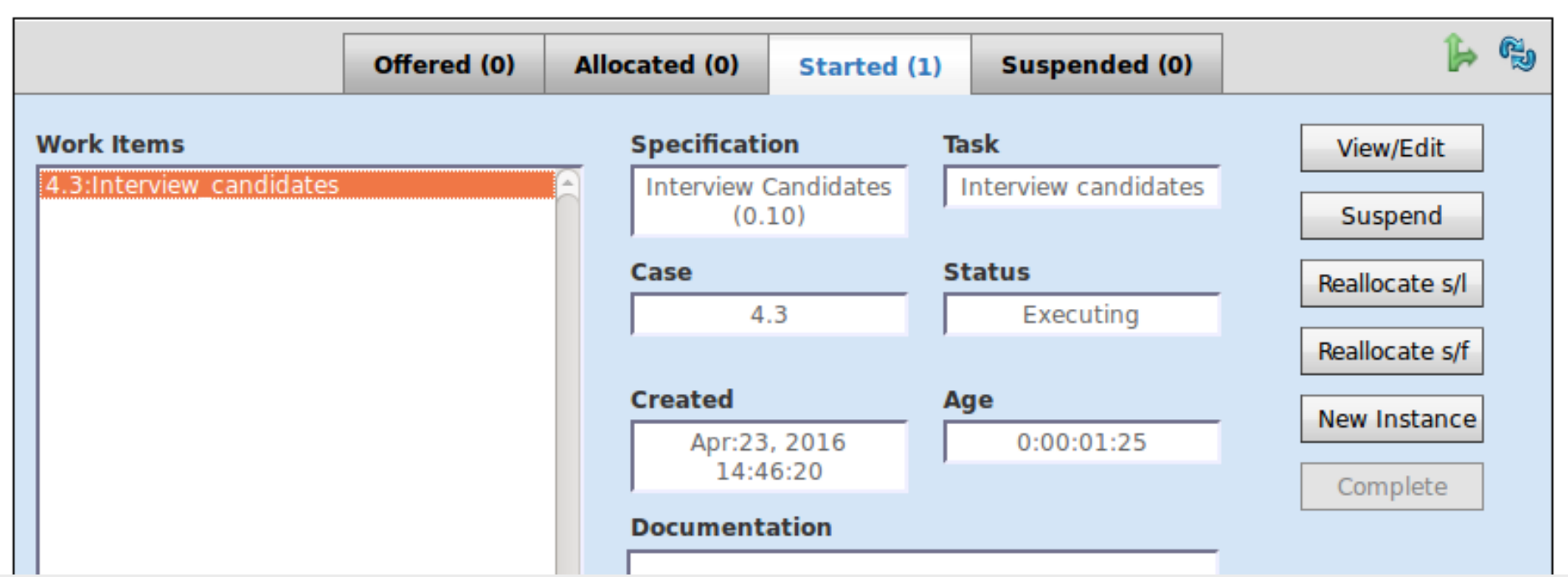
Minimum Die minimale Zahl von Instanzen, die bei der Aktivierung gestartet werden.

Maximum Die maximale Zahl von Instanzen, die bei der Aktivierung gestartet werden kann.

Threshold Wenn die Zahl der beendeten Instanzen die Schwelle überschritten haben oder alle gestarteten beendet sind, wird die nachfolgende Stelle aktiviert.

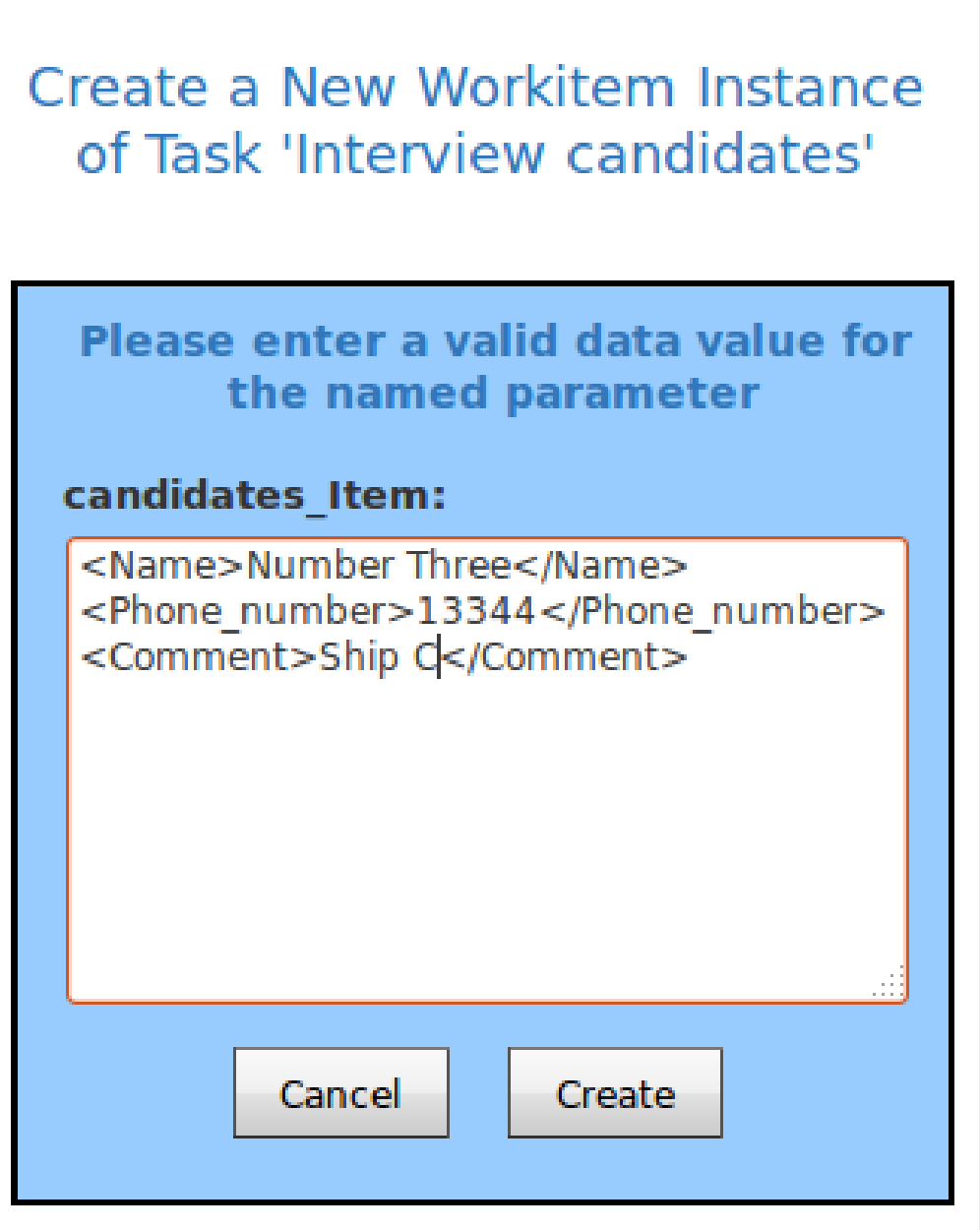
Dynamic instance creation Wenn dies gesetzt ist, erscheint für den User im Control Centre während Instanzen aktiv sind, ein Button "New instance", mit dem weitere Instanzen erzeugt werden können.

Multiple Instance Attribute



[Hen12]

Multiple Instance Attribute

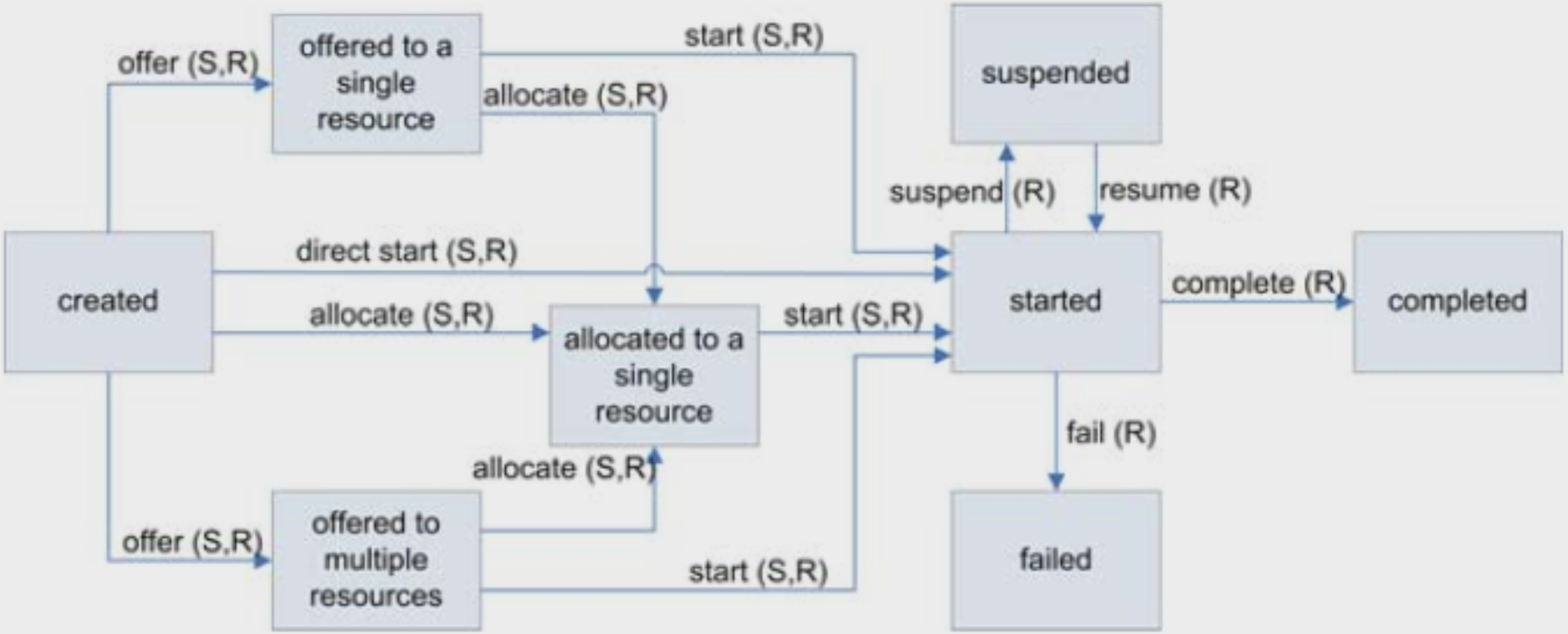


[Hen12]

Yet Another Workflow Language

Ressourcen Perspektive

YAWL-Lifecycle



[HAAR10]

Ressource (R) bzw. System (S) initiiert Zustandsübergang.

Creation Patterns I

Zur Entwurfszeit wird überlegt, wer zur Ausführungszeit die Work-Items ausführen soll.

Die folgenden Creation Patterns und ihre Umsetzung mit dem YAWL-Editor werden in YAWL Chapter 2 Part 4, The Workflow Resource Patterns (chapter02_part4.ppt) 55 -108 beschrieben.

Direct Allocation Die Ressource wird zur Ausführungszeit direkt benannt. Dies ist möglich, sollte jedoch nur in Ausnahmefällen benutzt werden, da man sonst bei einem Mitarbeiterwechsel die Workflows anpassen muss.

Role-Based Allocation Man weist das Work-Item einer Rolle zu. Wer das Work-Item tatsächlich bekommt, wird zur Ausführungszeit nach verschiedenen Strategien bestimmt.

Deferred Allocation Man kann die Ressourcen, die das WI ausführen sollen, auch in einer Variablen speichern. Auf diese Weise wird die Entscheidung, wer das WI ausführt, komplett zur Ausführungszeit getroffen.

Creation Patterns II

Separation of Duties Dies ist das Vier-Augen-Prinzip. Man kann festlegen, dass ein WI in einem Prozess nicht von derselben Ressource wie ein vorhergehendes durchgeführt werden soll.

Retain Familiar Man kann festlegen, dass ein WI in einem Prozess von derselben Ressource wie ein vorhergehendes durchgeführt werden soll.

Capability-Based Allocation Zusätzlich zu Rollen gibt es noch Fähigkeiten. Nach letzteren kann zusätzlich gefiltert werden.

Push Patterns I

Hier leitet das System neu erzeugte Work-Items proaktiv an Ressourcen weiter.

Die folgenden Push Patterns und ihre Umsetzung mit dem YAWL-Editor werden in YAWL Chapter 2 Part 4, The Workflow Resource Patterns (chapter02_part4.ppt) 55 -108 beschrieben.

Distribution by Offer - Single Resource Die Ressource kann "sich selbst überlegen", ob sie das WI ausführen möchte.

Distribution by Offer - Multiple Ressources Hier wird in einer Gruppe von Ressourcen "ein Freiwilliger" gesucht.

Distribution by Allocation - Single Resource Hier wird das WI einer Ressource fest zugewiesen, die es dann auch bearbeiten sollte. Dabei gibt es verschiedene Verteilungsstrategien: Shortest Queue, Random Allocation, Round Robin Allocation.

Pull Patterns I

Hier suchen sich die Ressourcen selbst die Work-Items.

Die folgenden Pull Patterns und ihre Umsetzung mit dem YAWL-Editor werden in YAWL Chapter 2 Part 4, The Workflow Resource Patterns (chapter02_part4.ppt) 55 -108 beschrieben.

Resource-Initiated Allocation Die Ressource kann die Absicht erklären, das WI bearbeiten zu wollen, jedoch ohne die Arbeit daran sofort zu beginnen. Auf diese Weise steht es zunächst anderen Ressourcen nicht mehr zur Verfügung.

Resource-Initiated Execution – Allocated Work Item Ein auf diese Weise zugewiesenes WI kann dann später gestartet werden.

Weitere Patterns in: YAWL Chapter 2 Part 4, The Workflow Resource Patterns (chapter02_part4.ppt) 90 -108.

Fragen

Live Feedback

- In einem Finanzamt gibt es in der Steuerabteilung drei Teams von je fünf Sachbearbeitern, die jeweils für einen Bereich der Fälle abhängig vom Anfangsbuchstaben des Nachnamens zuständig sind. Welche der Strategien würden Sie wählen?

Resource Management I

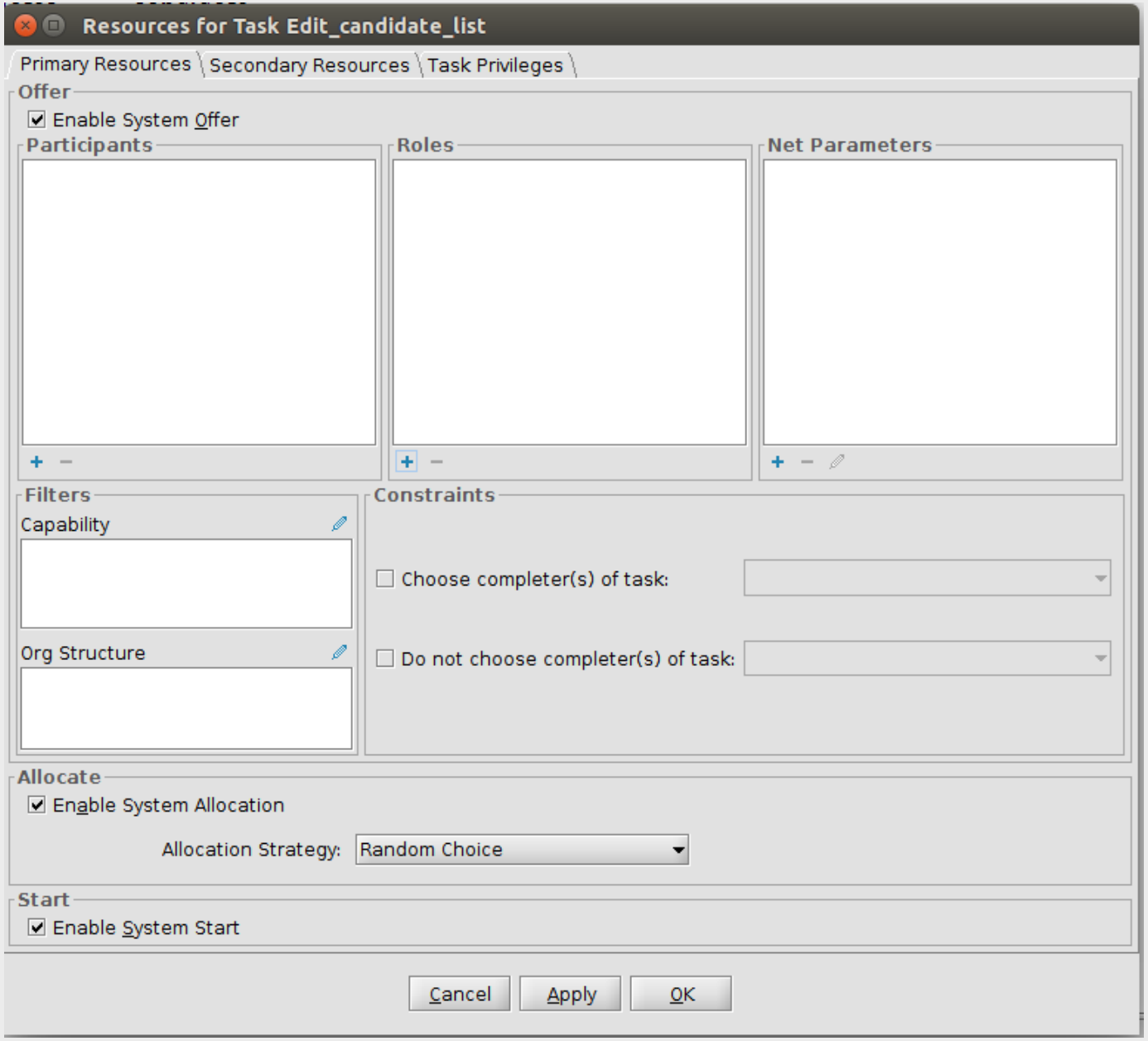
Unter dem Punkt Resourcing einer Task kann man einstellen, welche der drei Punkte Offer, Allocate, Start vom System (S) oder vom User (U) durchgeführt werden.

Offer	User	Ein Administrator muss einen Teilnehmer aussuchen, dem er das Work Item anbietet.
	System	Hier bietet das System das Work Item den Teilnehmern oder Rollen an. Außerdem können verschiedene Filter gesetzt werden.
Allocate	User	Wenn dies auf User und Offer auf User gesetzt sind, so bietet der Admin das Work Item einem Teilnehmer an, und danach weist sich der Teilnehmer dies selbst zu bzw. der Admin weist es noch einmal jemandem anderes zu.

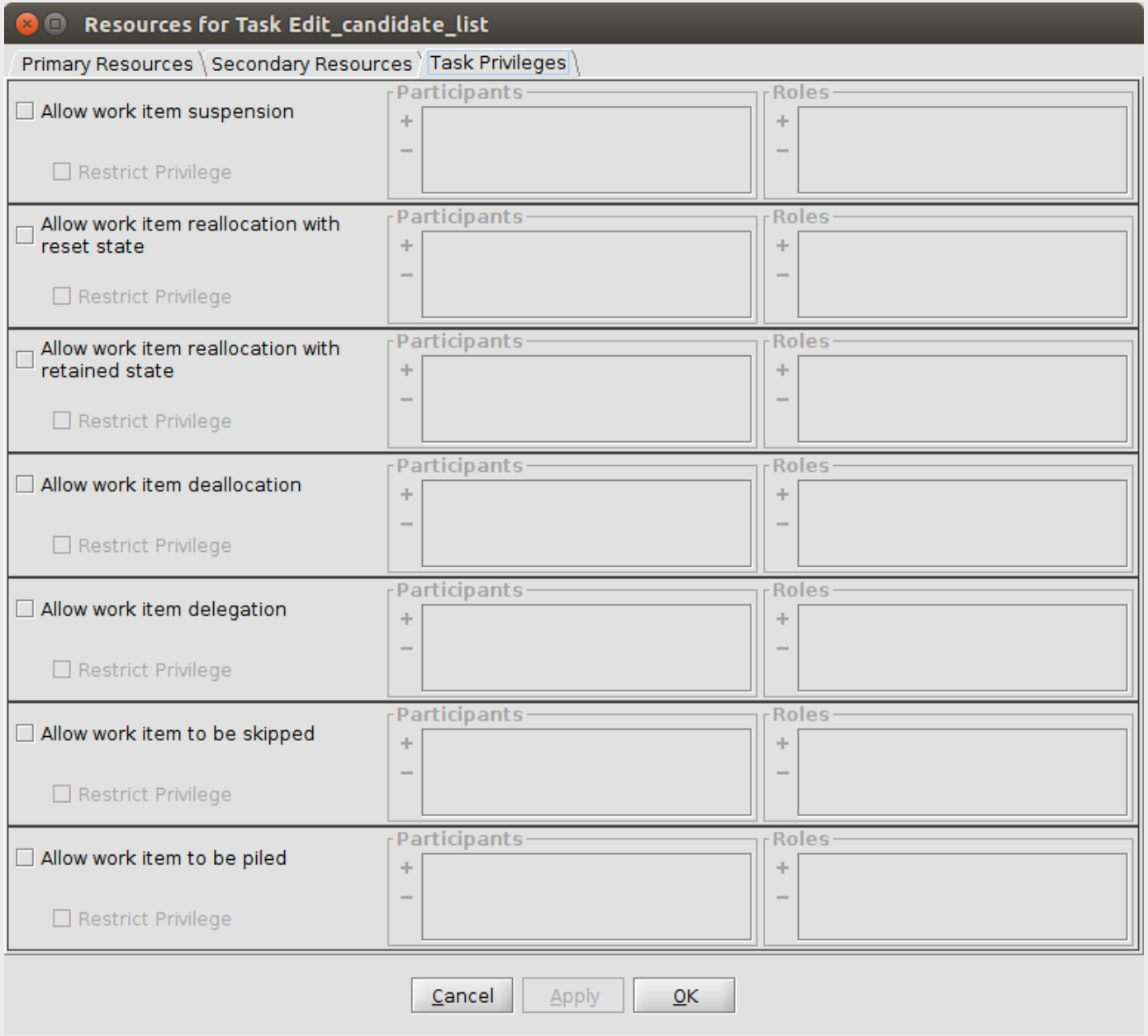
Resource Management II

	System	Wenn dies auf System und Offer auf System gesetzt sind, so wird das Work Item einem Teilnehmer automatisch zugewiesen. Dabei kann man zwischen verschiedenen Strategien wählen.
Start	User	Der Teilnehmer startet das Work Item selbst.
	System	Das Work Item ist bereits gestartet.

Resource Dialog



Task Privilegien



Task Privilegien I

In einem weiteren Tab desselben Dialoges kann man Task-Privilegien vergeben:

suspend Im Resource Service Schirm kann der Teilnehmer das Work Item dann auf suspended setzen. Das WI erscheint dann in einer gesonderten Liste, aus der man es wieder starten kann.

reallocate resetting state Wenn das WI gestartet ist, erscheint Reallocate s/l und man kann es anderen Teilnehmern zuweisen. Dies setzt allerdings voraus, dass man das Privileg View All Work Items of Team hat. Sonst kommt die Meldung "there are no participants that you are allowed to allocate to".

reallocate retaining state Dito mit Reallocate s/f

deallocate Ein Zurückweisen des WI wird ermöglicht. Wenn die Zuweisungsstrategie automatisch (system) eingestellt ist, so wird das WI sofort einem anderen zugewiesen.

Task Privilegien II

delegate Ein zugewiesenes WI kann an von der Position her untergeordnete zugewiesen (delegiert) werden – ein gestartetes nicht.

skip Danach ist das Workitem im Status complete aber es sind dann z. B. keine Pflichtdatenfelder gefüllt. Diese Option ist mit Vorsicht zu genießen.

pile Alle WI dieser Task werden direkt diesem Teilnehmer zugewiesen.

Organisation I

Role Eine Rolle ist eine Menge von Aufgaben, die von einer oder mehreren Teilnehmern eingenommen werden kann. Außerdem kann man Rollen und Subrollen bilden. So können etwa die Kostenstellenverantwortlichen eine Teilmenge aller Mitarbeiter sein ("belongs to" unter OrgData). Es ist sinnvoll die Rolle aller Mitarbeiter anzulegen, da man so immer allen ein Workitem anbieten kann.

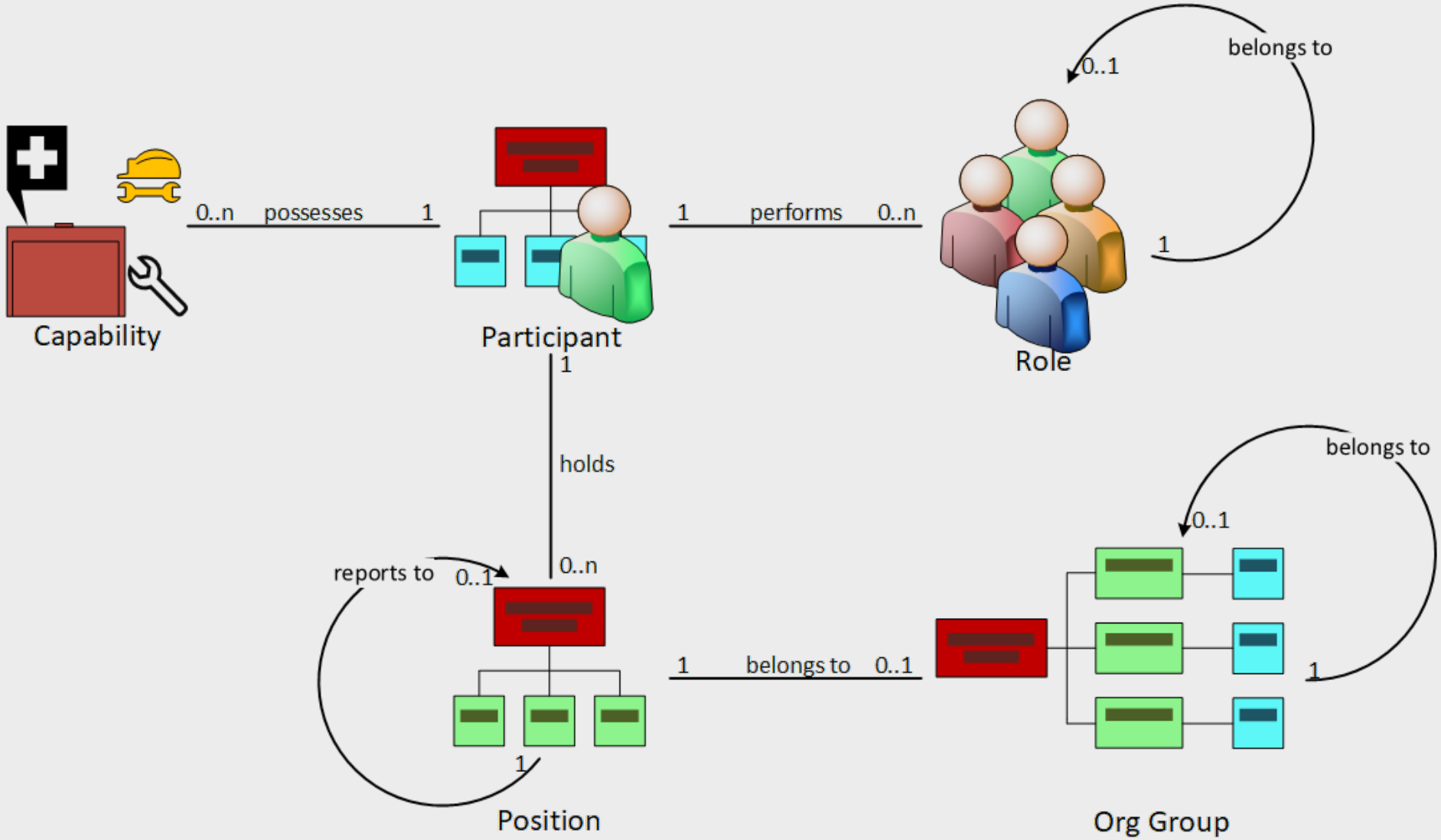
Capability Eine Capability ist eine Fähigkeit, die ein Teilnehmer besitzen kann (z. B. Bachelor of Science, Führerschein Klasse A). Nach Fähigkeiten wird zusätzlich gefiltert.

Organisation II

Position Normalerweise wird eine Position von einem Teilnehmer eingenommen. Eine Position kann zu null oder einer anderen Position berichten ("reports to"). Damit kann man die typischen Organigramme aufbauen. Eine Position kann zu null oder mehreren OrgGroups gehören.

Org Group Eine Org Group ist eine funktionale Gruppe von Positionen. Eine Position kann zu null oder einer Org Group gehören. Org Groups können ineinander geschachtelt sein.

Organisational Model



[HAAR10]

Yet Another Workflow Language

Resource Service

Komponenten I

YAWL Chapter 10 The Resource Service (chapter10.ppt)

Der Resource Service hat die Aufgabe, den Work Items Ressourcen zuzuweisen. Er besteht aus folgenden Komponenten:

Resource Manager arbeitet alle Resource Patterns ab.

- **Worklist** Web-basierte Benutzerschnittstelle.

Forms Connector dieser zeigt entweder dynamisch generierte Formulare oder Custom Forms an.

Codelet Server führt Codelets aus.

Arbeitsweise

- Zur Laufzeit erhält der Resource Service für alle WI, die nicht einem anderen YAWL Service zugeordnet werden, EnabledWorkItemEvent Nachrichten von der Engine über Interface B.
- Für manuelle Tasks wird aus der Spezifikation das initiale Distribution Set berechnet und anschließend die verschiedenen Filter und Allokationsstrategien angewandt.
- Für automatisierte Tasks wird das entsprechende Codelet ausgewählt.

Interfaces I

Mit der Engine kommuniziert der Resource Service über Interfaces A, B und E. Außerdem hat er folgende eigene Interfaces:

- **Interface O** für externe organisatorische Datenquellen. Diese können direkt vom Service aufgerufen werden.
- **Interface R** Zugriff für autorisierte Clients auf die eigenen organisatorischen Daten (z. B. für den YAWL Editor).
- **Interface W** stellt die gesamte Worklist zur Verfügung, so dass externe, spezialisierte Worklists entwickelt werden können.

Yet Another Workflow Language

Methode zum Erstellen eines Workflows

1. Informationen sammeln
2. Datentypen festlegen
3. Netzvariablen anlegen
4. Tasks erstellen
5. Ressourcen zuweisen

1) Informationen sammeln I

Als Beispiel dient die Fallstudie "Procure to Pay" [DRMR13, HM15]. Es macht Sinn, Aussagen zu den drei Perspektiven im Text farblich zu markieren.

Kontrollfluss
- Wodurch wird ein Fall bestimmt?
- Finde Elemente, die als Tasks geeignet sind.
- Gibt es Aussagen über die Reihenfolge?

Ressourcen
- Finde alle Rollen, die an dem Prozess beteiligt sind.
- Wenn konkrete Personen genannt werden, versuche die Rolle zu ermitteln.

Daten
- Was wird über verarbeitete Informationen oder Daten ausgesagt?
- Kann man zunächst einfache Annahmen treffen?

2) Datentypen festlegen I

Häufig gebrauchte Konstrukte:

Enumeration

```
<xs:simpleType name="maritalStatusType">
    <xs:restriction base="xs:string">
        <xs:enumeration value="Single" />
        <xs:enumeration value="Married" />
        <xs:enumeration value="Divorced" />
        <xs:enumeration value="Widow" />
    </xs:restriction>
</xs:simpleType>
```

2) Datentypen festlegen II

Record

```
<xs:complexType name="candidateType">
   <xs:sequence>
      <xs:element name="Name" type="xs:string"/>
      <xs:element name="Phone_number" type="xs:string"/>
      <xs:element name="Comment" type="xs:string"/>
   </xs:sequence>
</xs:complexType>
```

2) Datentypen festlegen III

List

```
<xs:complexType name="candidatesType">
   <xs:sequence>
      <xs:element name="Candidate" type="candidateType"
          minOccurs="1" maxOccurs="unbounded"/>
   </xs:sequence>
</xs:complexType>
```

3) Netzvariablen anlegen I

Typ

- Normalerweise Local. Variablen mit komplexen Typen sollten immer initialisiert werden.
- Variablen vom Typ Input werden beim Start des Workflow als Parameter mitgegeben.
- Variablen vom Typ Output werden nach dem Ende an den Aufrufer zurückgegeben – interessant bei Worklets.

Subnetze

- Subnetze sollten i. d. R. erst später erstellt werden.
- Aufruf von anderen Workflows über Worklets möglich (s. `http://yaug.org/content/reusing-yawl-specifications-subworkflows` [Hen12]).
- Benötigte Variablen müssen an Subnetze übergeben werden.

3) Netzvariablen anlegen II

Kontrolle

- Nutzung von Netzvariablen kann im Editor mit # kontrolliert werden.

4) Tasks erstellen I

Daten
- Benötigte Netzvariablen per Drag-and-Drop einziehen.
- Input/Output Verhalten festlegen.

Split Predicates
- Ggf. zusätzliche boolesche Netzvariablen festlegen.
- Diese in der Split-Task bearbeiten.
- Prädikate auf Basis der Netzvariablen definieren.

Subnetze
- Sinnvolle und abgeschlossene Bereiche als Subnetz definieren, um die Übersichtlichkeit zu erhöhen.

5) Ressourcen zuweisen I

Organisationsdaten

- Rollen und User im Control Centre anlegen und .ybkp-Datei exportieren.

Tasks bearbeiten

- Rollen zu Tasks zuweisen.
- Push/Pull Patterns berücksichtigen.
- Retain familiar und Vieraugenprinzip beachten.

Literatur I

Aalst, Wil van d. ; Adams, Michael ; Hofstede, Arthur ter ; Russel, Nick:
YAWL.
www.yawlfoundation.org.
Version: 2009

Aalst, W. M. P. d.:
The Application of Petri Nets to Workflow Management.
In: *The Journal of Circuits, Systems and Computers* 8 (1998), Nr. 1, 66, 21.
http://wwwis.win.tue.nl/wsinwa/Publications/p53.PDF

Aalst, Wil van d. ; Hofstede, A.H.M. ter:
YAWL: yet another workflow language.
In: *Information systems.* 30 (2005), Nr. 4, S. 245–275. – ISSN 0306–4379

Literatur II

Aalst, Wim van d. ; Hofstede, Arthur ter:
Workflow Patterns Home Page.
http://www.workflowpatterns.com/.
Version: 2010. –
http://www.workflowpatterns.com/

Adams, Michael ; Hense, Andreas V. ; Hofstede, Arthur H. M. t.:
Extensible ontology-based views for business process models.
In: *Knowledge and Information Systems* 63 (2021), Oktober, Nr. 10, 2763–2789.
http://dx.doi.org/10.1007/s10115-021-01604-1. –
DOI 10.1007/s10115–021–01604–1. –
ISSN 0219–3116

Literatur III

Clark, James ; DeRose, Steve:
XML Path Language (XPath) Version 1.0.
https://www.w3.org/TR/xpath/#section-Location-Steps.
Version: November 1999

Dumas, Marlon ; Rosa, Marcello L. ; Mendling, Jan ; Reijers, Hajo A.:
Fundamentals of Business Process Management.
Springer Berlin Heidelberg, 2013 http://link.springer.com/chapter/10.1007/978-3-642-33143-5_1. – ISBN 978–3–642–33142–8 978–3–642–33143–5

Hofstede, Arthur ter ; Adams, Michael:
YAWL - User Manual 2.1 beta.
The YAWL Foundation, 2010 http://www.yawlfoundation.org

Literatur IV

Hofstede, Arthur H. M. ; Aalst, Wil M. P. d. ; Adams, Michael ; Russell, Nick:
Modern Business Process Automation: YAWL and its Support Environment.
Springer, Berlin, 2010. –
ISBN 3–642–03120–X

Hense, Andreas V.:
YAWL User Group.
http://www.yaug.org.
Version: 2012. –
http://www.yaug.org

Literatur V

Hense, Andreas V. ; Malz, Robert:
Comparison of the Subject-oriented and the Petri Net Based Approach for Business Process Automation.
In: *Proceedings of the 7th International Conference on Subject-Oriented Business Process Management*.
Kiel, Germany : ACM, 2015 (S-BPM ONE '15). – ISBN 978–1–4503–3312–2, 21:1–21:12

Kiepuszewski, B. ; Hofstede, Arthur H. M. ; Aalst, W. M P. d.:
Fundamentals of Control Flow in Workflows.
In: *Acta Informatica* 39 (2002), S. 143–209

Literatur VI

KINDLER, Ekkart:
On the semantics of EPCs: Resolving the vicious circle.
In: *Data & Knowledge Engineering* 56 (2006), Januar, Nr. 1, 23–40.
http://dx.doi.org/10.1016/j.datak.2005.02.005. –
DOI 10.1016/j.datak.2005.02.005. –
ISSN 0169–023X

LAWRENCE, Peter:
Workflow Handbook 1997.
1.
Wiley, 1997. –
ISBN 0–471–96947–8

LA ROSA, Marcello ; CLEMENS, Stephan ; HOFSTEDE, Arthur ter:
The Order Fulfillment Process Model.
yawlfoundation.org.
Version: September 2009

Literatur VII

Russell, Nick ; Aalst, Wil M. P. van d. ; Hofstede, Arthur H. M. t.:
Workflow Patterns: The Definitive Guide.
Cambridge, MA : The MIT Press, 2016. –
ISBN 978–0–262–02982–7

w3schools:
XPath Examples.
http://www.w3schools.com/xsl/xpath_examples.asp.
Version: 2013

w3schools:
XQuery Tutorial.
http://www.w3schools.com/xquery/.
Version: 2013

Literatur VIII

WFMC:
Workflow Management Coalition Terminology & Glossary. Bd. WFMC-TC-1011.
Workflow Management Coalition Specification, 1999
http://www.wfmc.org

Hinweise zur Nutzung dieses Foliensatzes

Dieser Foliensatz wird ausschließlich den Teilnehmern dieser Lehrveranstaltung zur Verfügung gestellt. Er dient als Hilfe für die Vor- und Nachbereitung der Inhalte der Lehrveranstaltung und zur Prüfungsvorbereitung. Für diese Zwecke darf eine lokale Kopie erzeugt werden. Dieser Foliensatz darf nicht an Dritte weitergegeben werden.

© Andreas V. Hense 2025